机器学习导论

第4章 深度学习与MindSpore

谢茂强

南开大学软件学院

主体摘自互联网，版本众多，未找到源头作者
关于MindSpore的介绍来自于华为昇腾社区公开课

近年深度学习取得突破的应用领域

- 语音识别：Hinton DBN+Microsoft 、CTC
- 计算机视觉：Hinton AlexNet、LeNet、vgg、ResNet
- 自然语言处理：RNN、LSTM、Encoder-Decoder、Transformer(Self-Attention、BERT、GPT、ChatGPT、GPT4)

深度学习

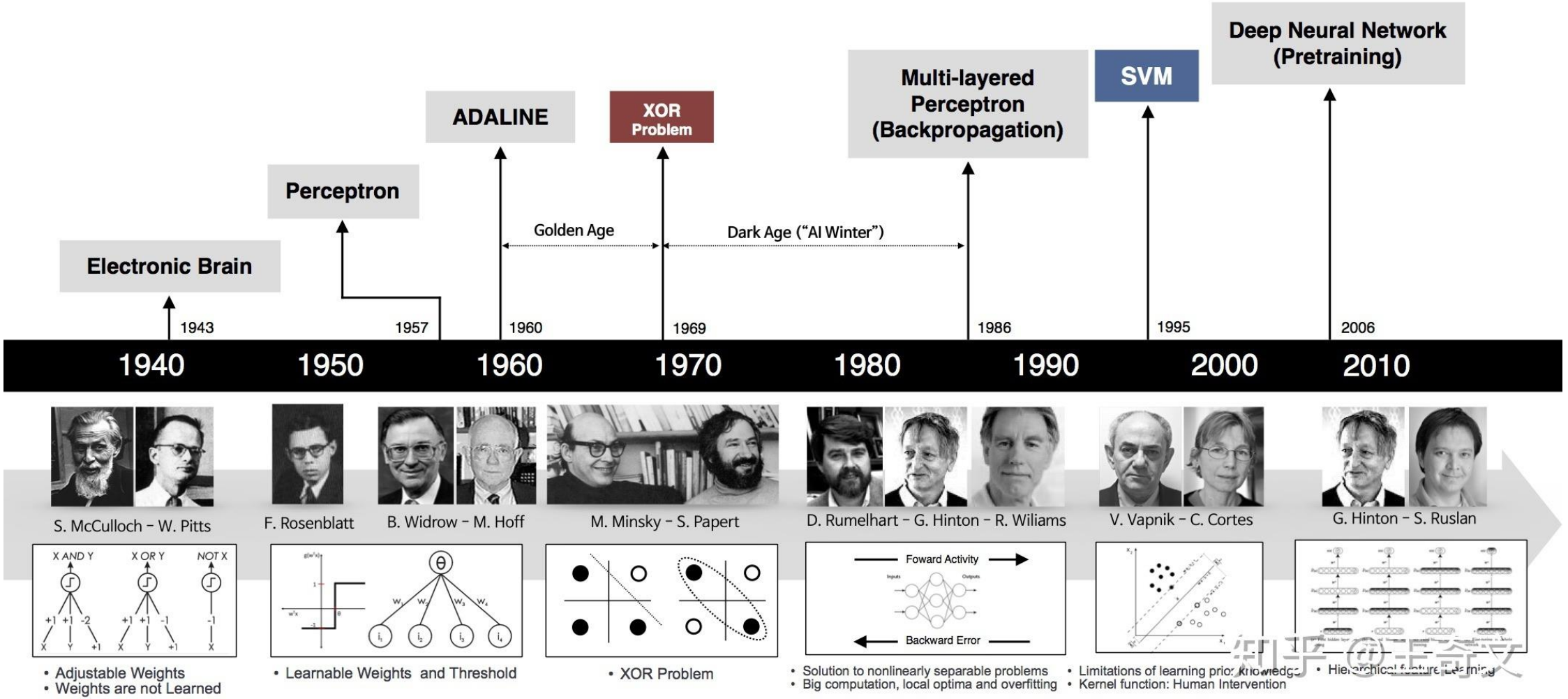
- Deep learning is a branch of machine learning based on a set of algorithms that attempt to model high level abstractions in data.
- The advantage of deep learning is to extracting features automatically instead of extracting features manually.

✓ Computer vision

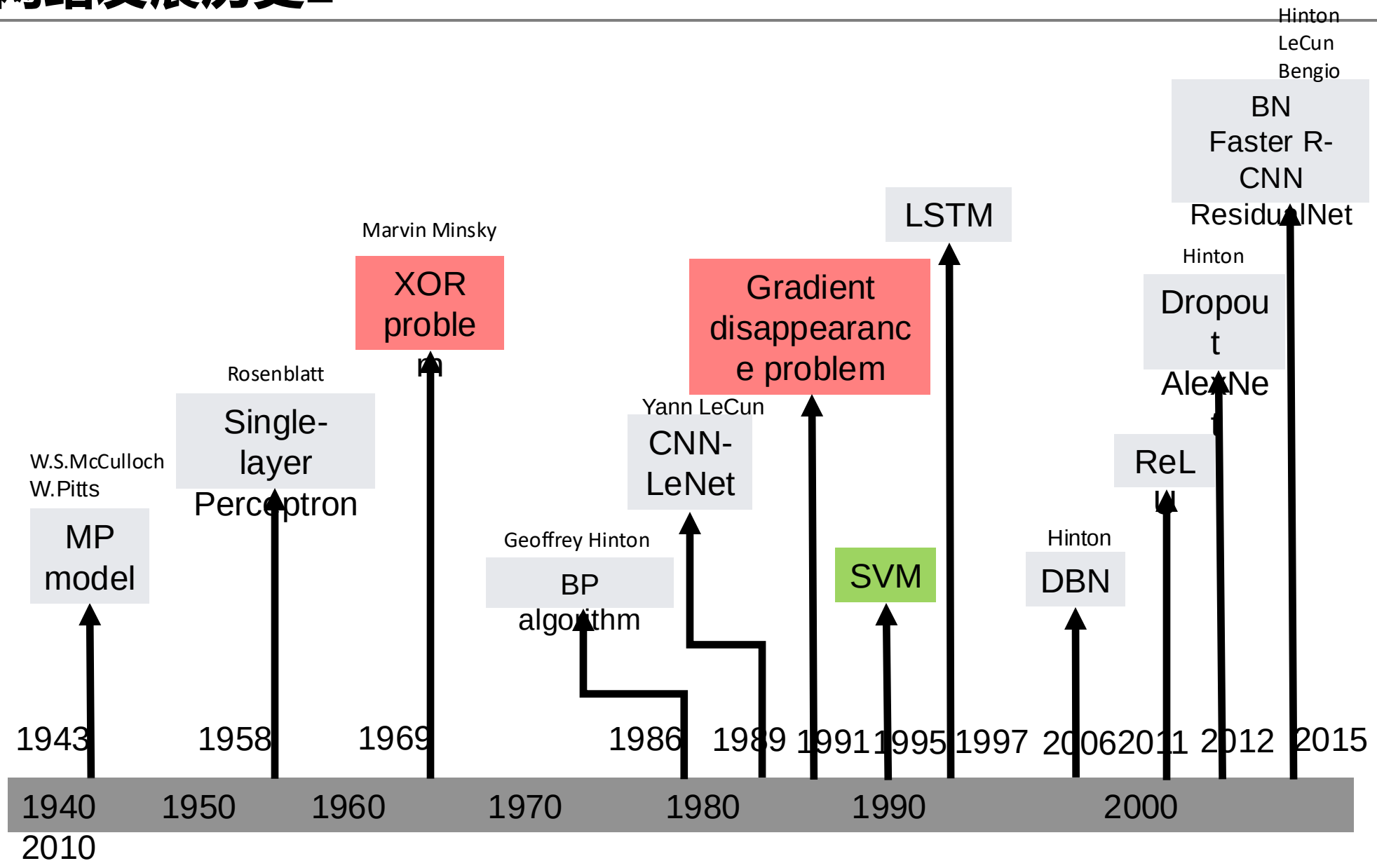
✓ Natural language processing

✓ Speech recognition

神经网络发展历史1

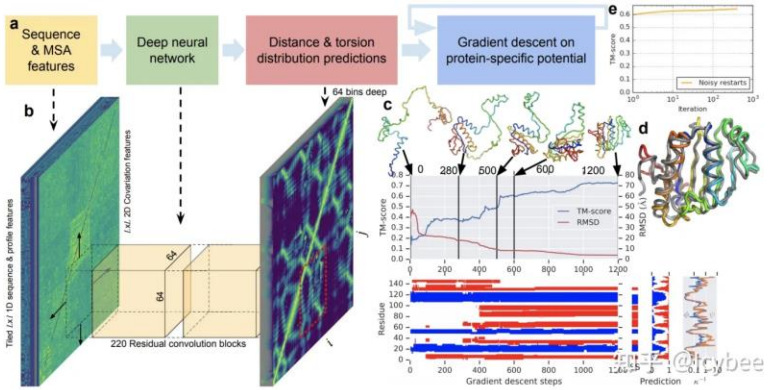

Deep Neural Network (Pretraining)
SVM
Multi-layered Perceptron (Backpropagation)
XOR Problem
ADALINE
Perceptron
Electronic Brain
Golden Age
Dark Age ("AI Winter")
1943
1957
1960
1969
1986
1995
2006
1940
1950
1960
1970
1980
1990
2000
2010
S. McCulloch - W. Pitts
F. Rosenblatt
B. Widrow - M. Hoff
M. Minsky - S. Papert
D. Rumelhart - G. Hinton - R. Wiliams
V. Vapnik - C. Cortes
G. Hinton - S. Ruslan
X AND Y
X OR Y
NOT X
Foward Activity
Backward Error
• Adjustable Weights
• Weights are not Learned
• Learnable Weights and Threshold
• XOR Problem
• Solution to nonlinearly separable problems
• Big computation, local optima and overfitting
• Limitations of learning prior knowledge
• Kernel function: Human Intervention
• Hierarchical feature learning

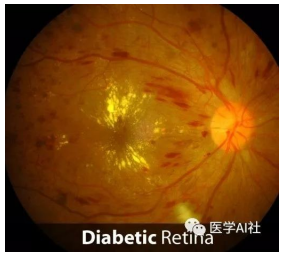
神经网络发展历史2


Hinton
LeCun
Bengio
BN
Faster R-CNN
ResidualNet
LSTM
Marvin Minsky
XOR proble
m
Hinton
Dropou
t
AlexNe
t
Gradient disappearanc
e problem
Rosenblatt
Single-
layer
Perceptron
Yann LeCun
CNN-
LeNet
ReL
U
W.S.McCulloch
W.Pitts
MP
model
Geoffrey Hinton
BP
algorithm
SVM
Hinton
DBN
1943
1958
1969
1986
1989
1991
1995
1997
2006
2011
2012
2015
1940
1950
1960
1970
1980
1990
2000
2010

第三次神经网络的崛起为什么是在2010年之后出现

- 更多的数据
  - 标注的文本、图片、视频、棋谱
- 更强的算力
  - 算力大幅提升
  - 专用加速芯片、





- 应用环境上
  - 足够多和便宜的传感器、网络
  - 足够多的有经验的工程师
  - 就差最后一步的机会
  - 互联网时代培育好的投融资环境

深度学习

- 主要是特征自动构建
- 计算机视觉：分类（是什么）、检测（是什么？在哪里？YOLO）、分割（每个像素属于那个目标或场景，分示例分割和语义分割，像素抠图）CNN
- 自然语言处理（序列任务）：中文分词、词性标注、命名实体标注、句对关系判断（知识图谱）、AutoQA、机器翻译、情感分析：RNN
- 语音识别与语音合成

计算机视觉：ImageNet

- Database: part of ImageNet database,

  1000 categories, 1.2 million training

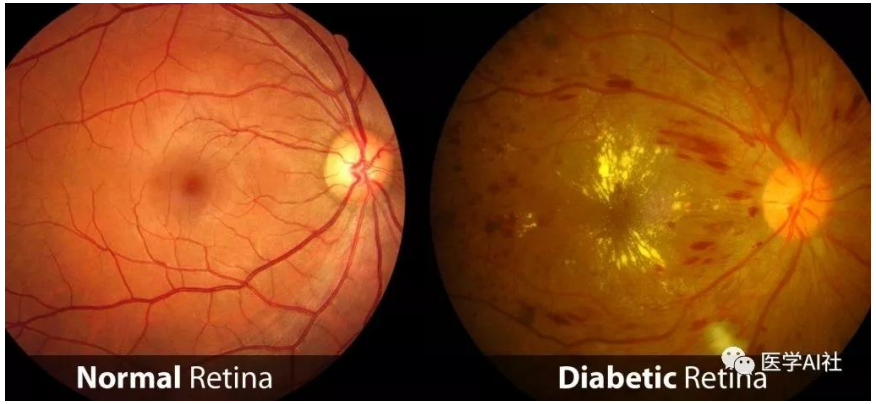
  images, 150,000 testing images.
- Task: classify testing image into one of

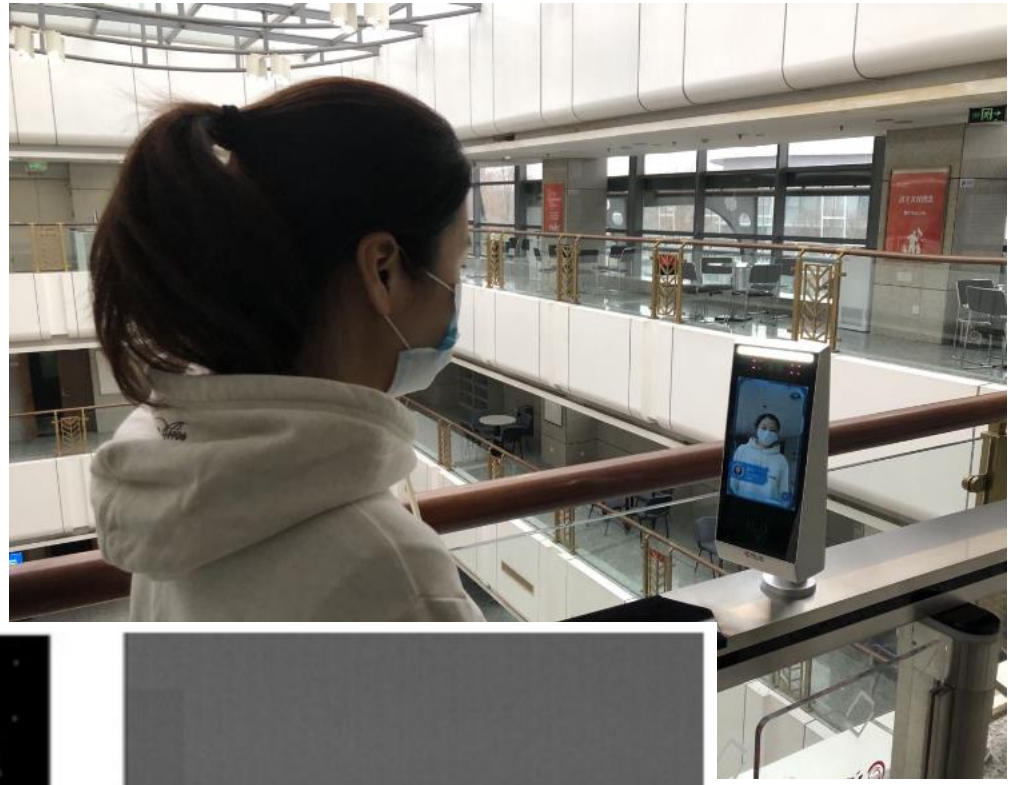
  1000 categories.
- Examples of two categories needed to be differentiated:

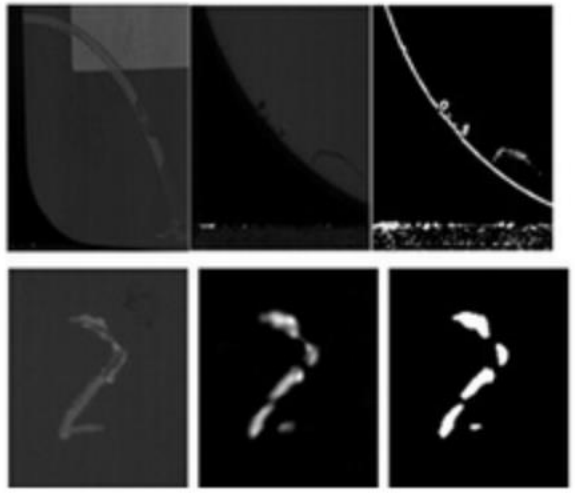
LI FeiFei @ Stanford Univ.

LI FeiFei. ImageNet: A Large-Scale Hierarchical Image Database. CVPR 2009 (Poster)

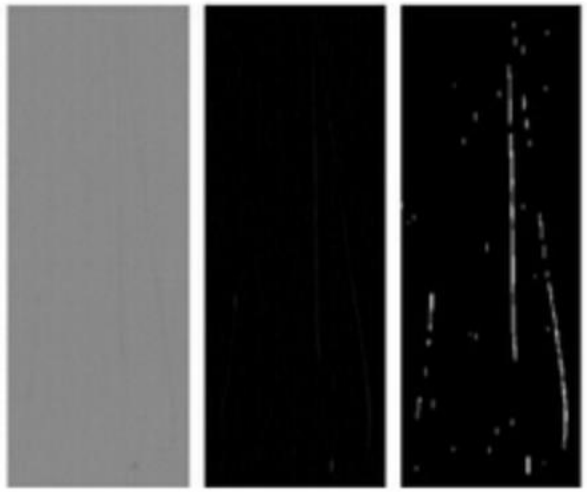
计算机视觉



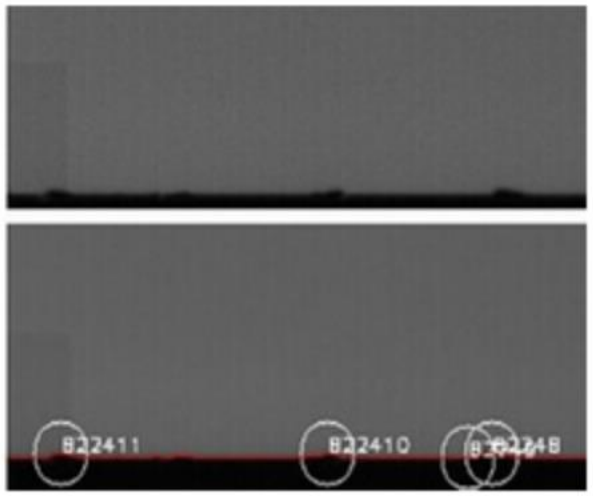




图一：边缘与印痕



图二：划痕与污点



图三：崩边缺陷定位检测

自然语言处理

- 基础任务
  - 文本向量化(Word Embedding)
  - 语言模型(Language Model)
- 应用
  - 文本分类：垃圾邮件分类、情感分析、意图识别等
  - 聚类：时间聚合、典型意见挖掘
  - 序列标注：命名实体识别、分词、词性标注、语义标注
  - 句对：QA、机器翻译
  - 生成式任务：机器翻译、文本摘要

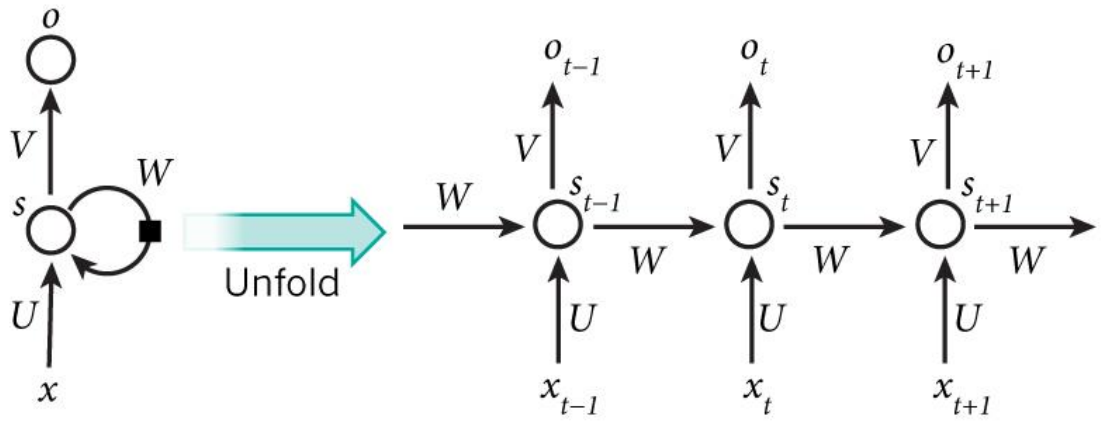
Deep Learning Frameworks

- Caffe、Mxnet、Keras
- Torch & PyTorch
- TensorFlow、华为MindSpore

自然语言处理(NLP)

- 自然语言处理（序列学习任务，Seq2Seq）：中文分词、词性标注 、句法标注、命名实体标注、句对关系判断（知识图谱）、AutoQA、机器翻译、情感分析：RNN
- 示例
  - 中文分词：长春市长春天下基层
    - 长春 市长 春天 下 基层
    - 长春市 长春 天下 基层
  - 翻译：今天我们来介绍自然语言处理的发展
    - Today we will introduce the development of natural language processing

RNN(Recurrent Neural Network)



What?

RNN aims to process the sequence data. RNN will remember the previous information and apply it to the calculation of the current output. That is, the nodes of the hidden layer are connected, and the input of the hidden layer includes not only the output of the input layer but also the output of the hidden layer.

How to train?

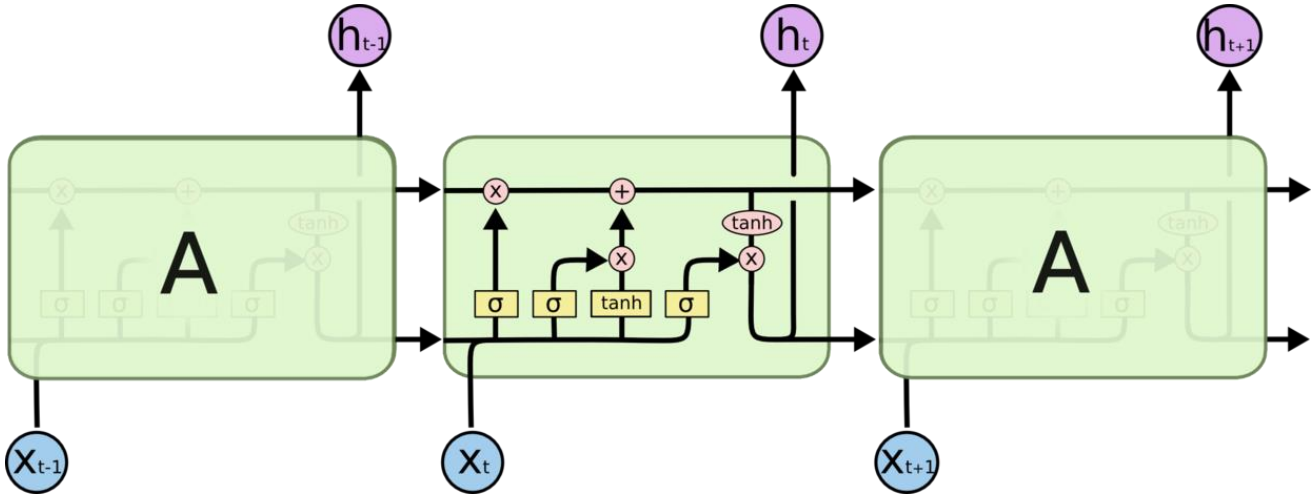
BPTT(Back propagation through time)

Applications?

- ✓ Machine Translation
- ✓ Generating Image Descriptions
- ✓ Speech Recognition

Marhon S A, Cameron C J F, Kremer S C. Recurrent Neural Networks[M]// Handbook on Neural Information Processing. 2013:29-65.

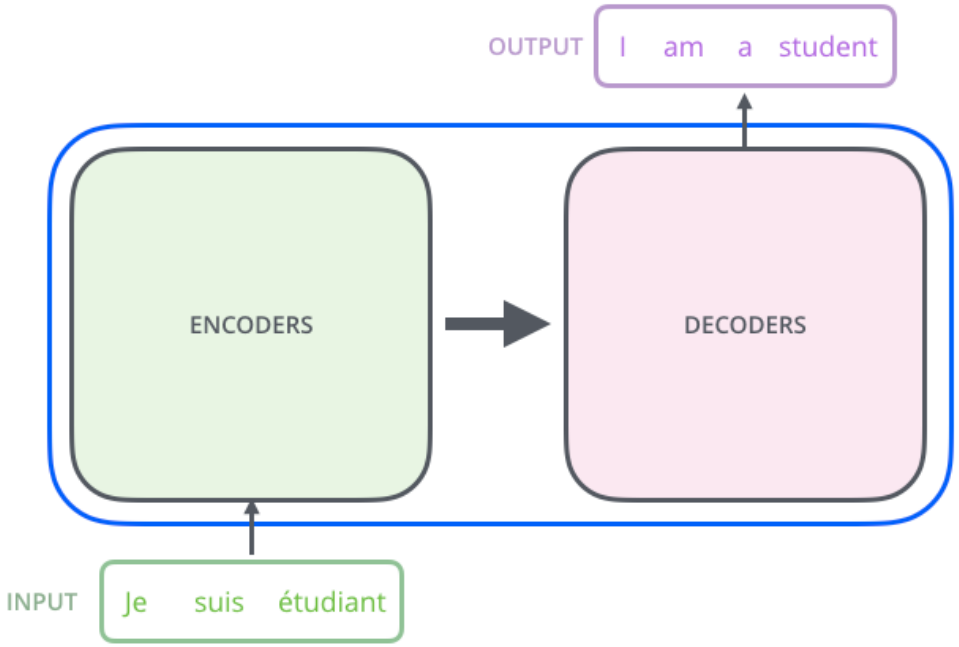
Long Short-Term Memory(LSTM,1997)



https://www.cnblogs.com/wangduo/p/6773601.html?utm_source=itdadao&utm_medium=referral

Transformer

- 2017年，Google提出，带有self multi-head attention的Encoder&Decoder



Transformer: Attention is all you need. Google. 2017

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Transformer & BERT

- Google提出、12层、相比于之前的Transformer，它更窄。宽度从2048到1024
- 用 unsupervised 的办法，来训练 transformer 模型。
  - 用数值向量（word vector）来表达自然语言词汇的语义
  - 如何给每个词汇，找到恰当的数值向量?
- Pre-training、Deep、Bidirectional、Transformer、Language Understanding

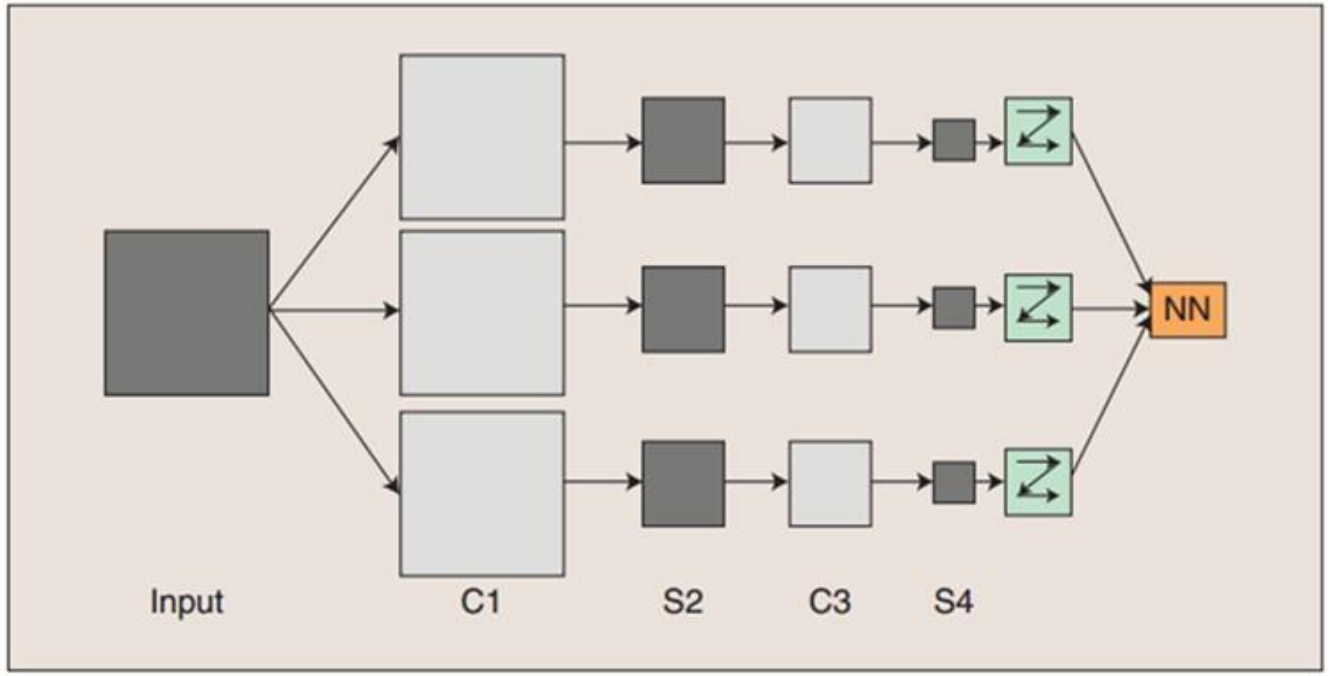
Transformer: Attention is all you need. Google. 2017
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

ChatGPT, GPT4 and AIGC

- OpenAI提出的Generative Pre-trained Transformer 4
- 新闻生成、翻译、问答、填空、同义词替换、简单计算等
  - 生成的新闻很难判断是否是由机器生成
- 1750亿个参数
- 5千亿单词预训练

卷积神经网络(Convolutional Neural Networks, CNN)

- ✓ Convolution neural network is a kind of feedforward neural network, which has the characteristics of simple structure, less training parameters and strong adaptability.
- ✓ CNN avoids the complex pre-processing of image(etc.extract the artificial features), we can directly input the original image.
- ✓ Basic components : Convolution Layers, Pooling Layers, Fully connected Layers



卷积层的示意

1_{x1}	1_{x0}	1_{x1}	0	0
0_{x0}	1_{x1}	1_{x0}	1	0
0_{x1}	0_{x0}	1_{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

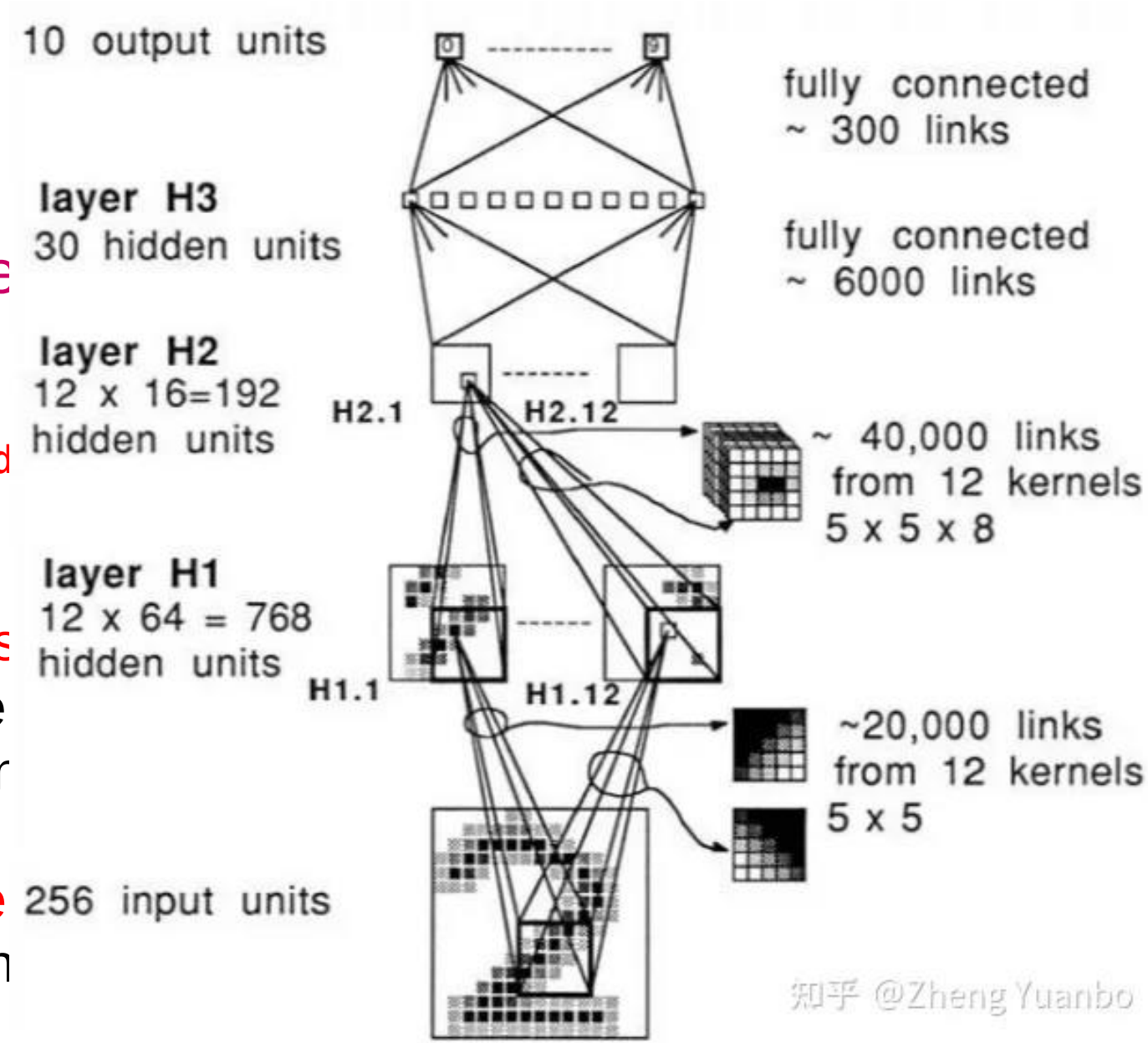
Convolved Feature

local

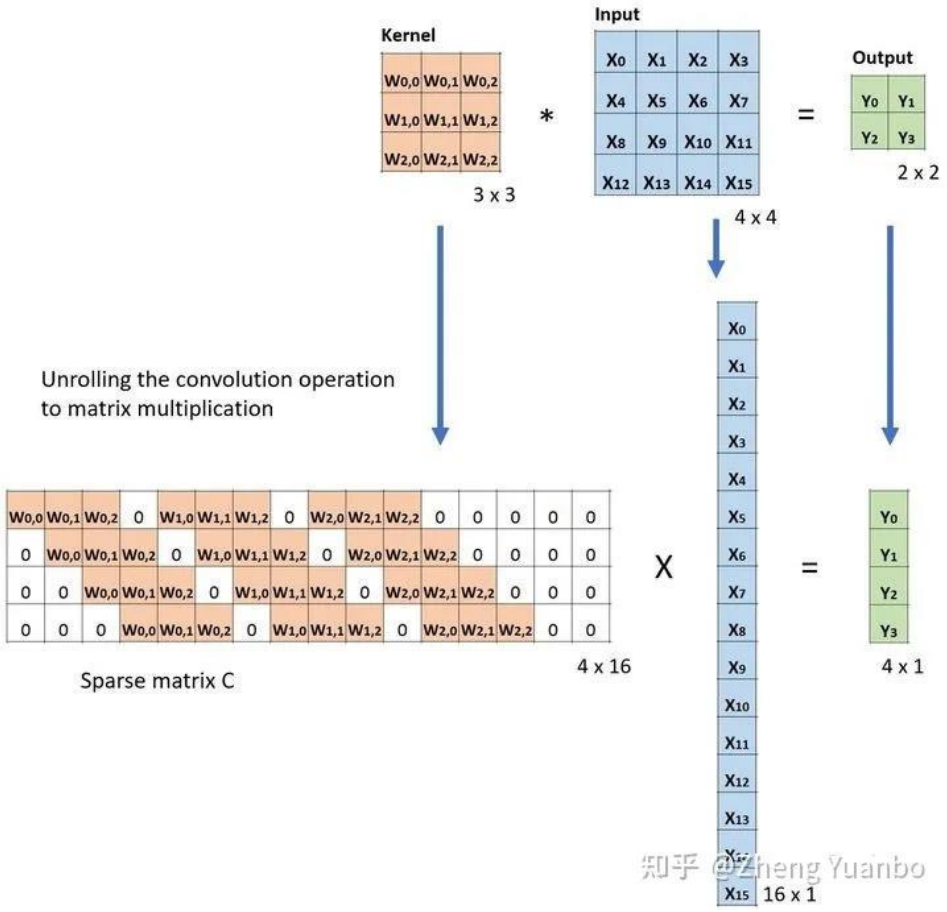
we

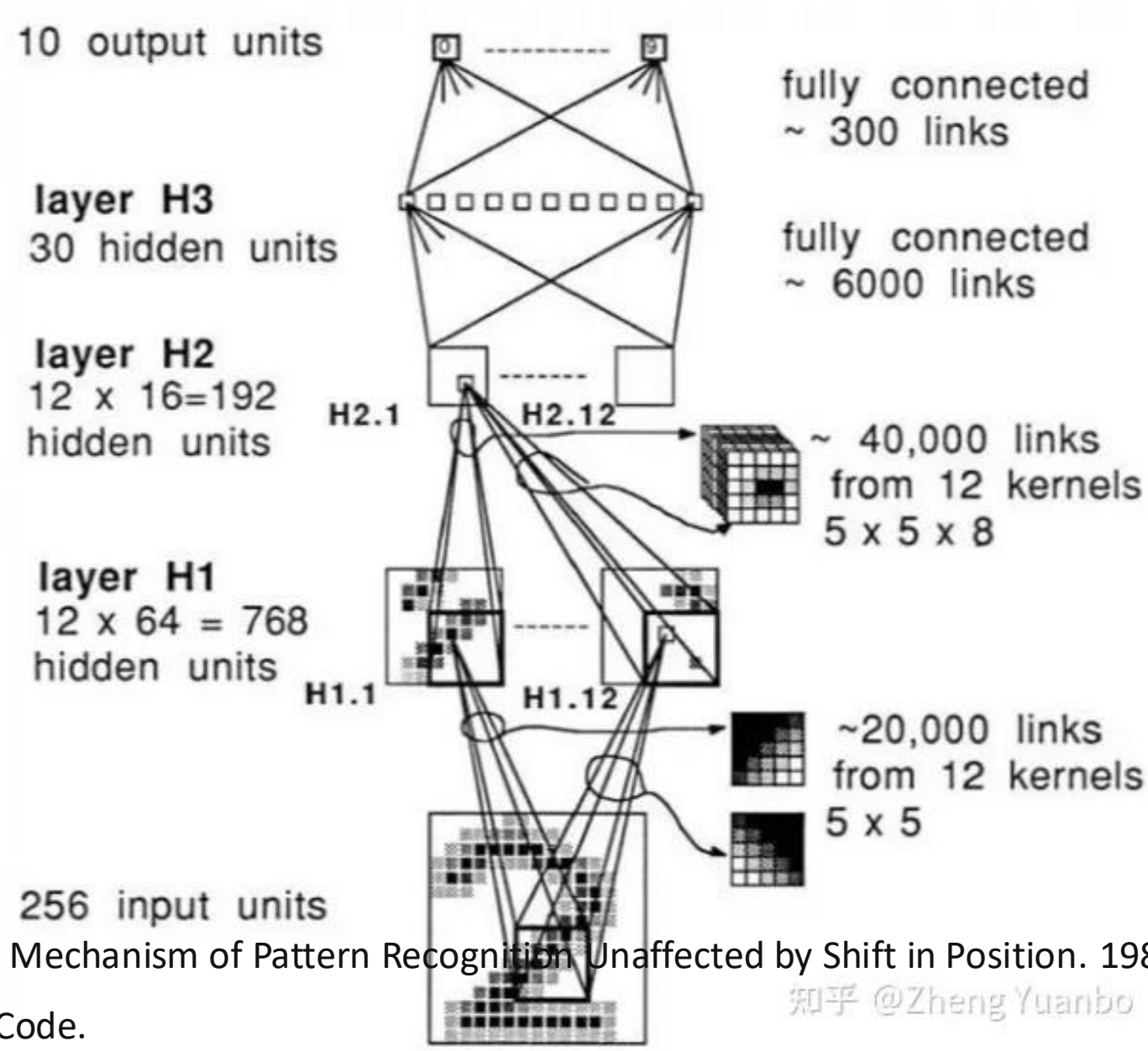
Reduced

- ✓ The convolution kernel translates element of the convolution kerne corresponding position of the cor product.
- ✓ By moving the convolution kerne the sum of the product of the con



卷积层:Neocognitron（1989）



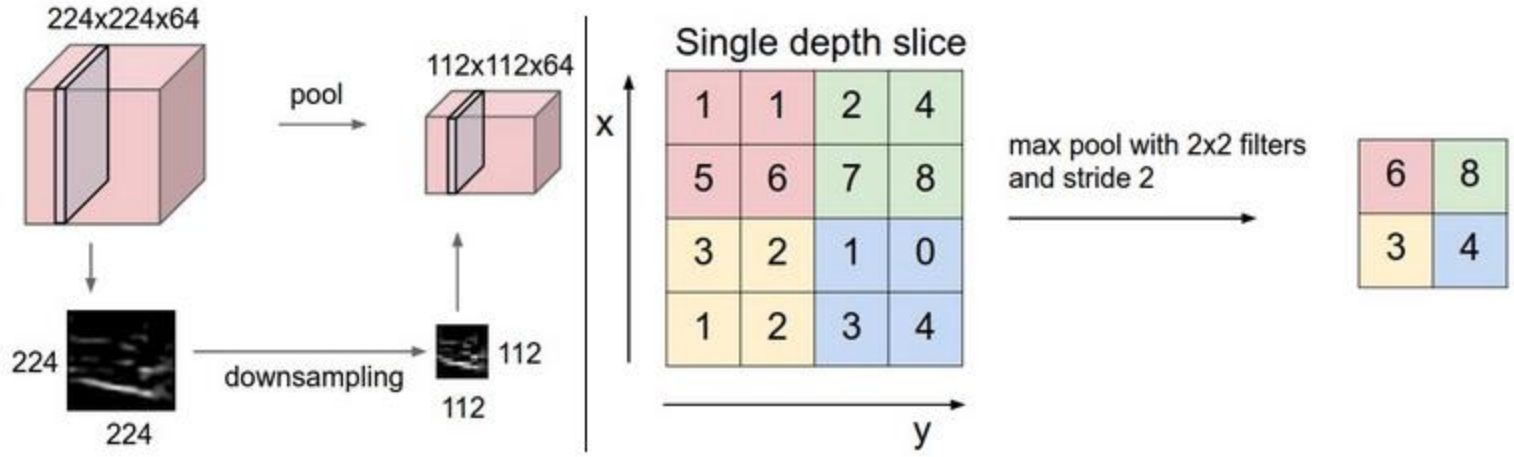


https://zhuanlan.zhihu.com/p/511088629

K. Fukishima. A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position. 198

LeCun, et al.. Backpropagation Applied to Handwritten Zip Code.

池化层(Pooling Layer)



Pooling layer aims to compress the input feature map（降维，特征压缩）, which can reduce the number of parameters in training process and the degree of over-fitting of the model.

Max-pooling : Selecting the maximum value in the pooling window.
Mean-pooling : Calculating the average of all values in the pooling window.

全连接层和SoftMax层
Fully connected layer and Softmax layer

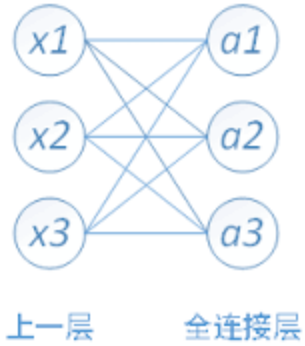


Fig1. Fully connected layer.

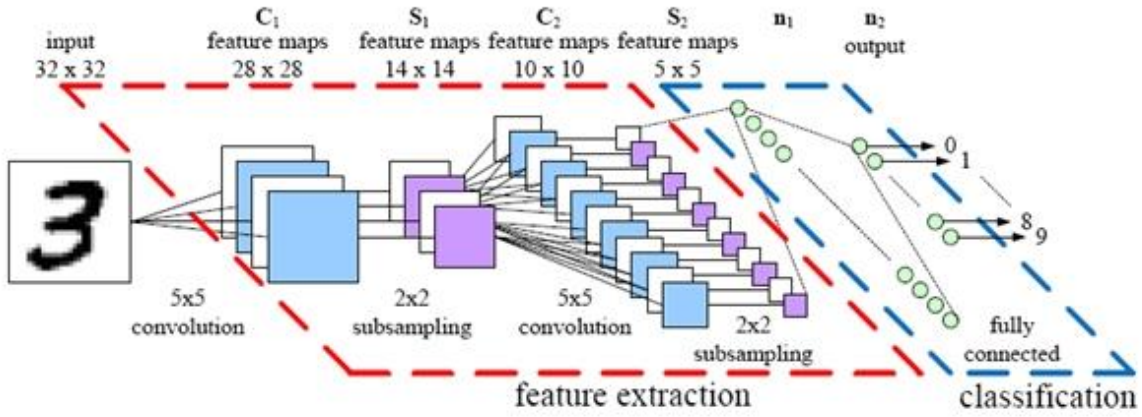


Fig2. Complete CNN structure.

Each node of the fully connected layer is connected to all the nodes of the last layer, which is used to combine the features extracted from the front layers.

$$f(z_j) = \frac{e^{z_j}}{\sum_{i=1}^{n} e^{z_i}}$$

分类器

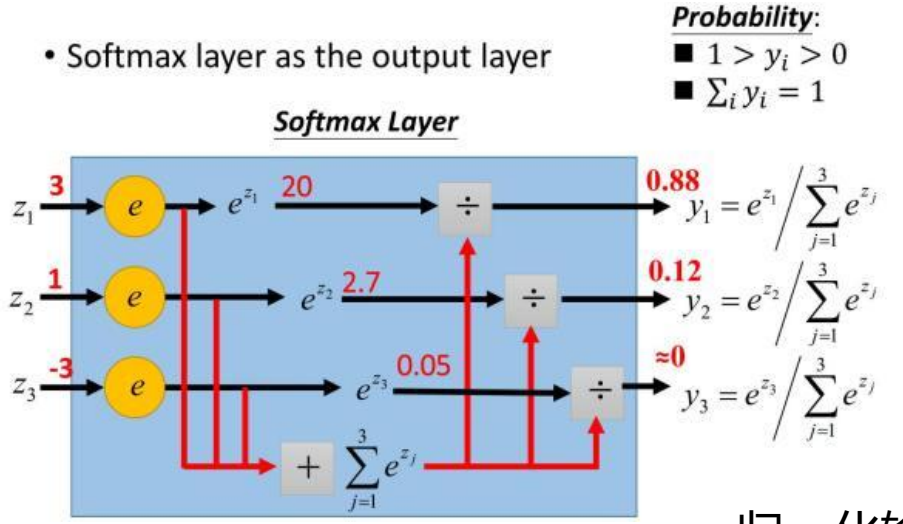


Fig3. Softmax layer.

归一化输出

CNN的训练和测试

Before the training stage, we should use some different small random numbers to initialize weights.

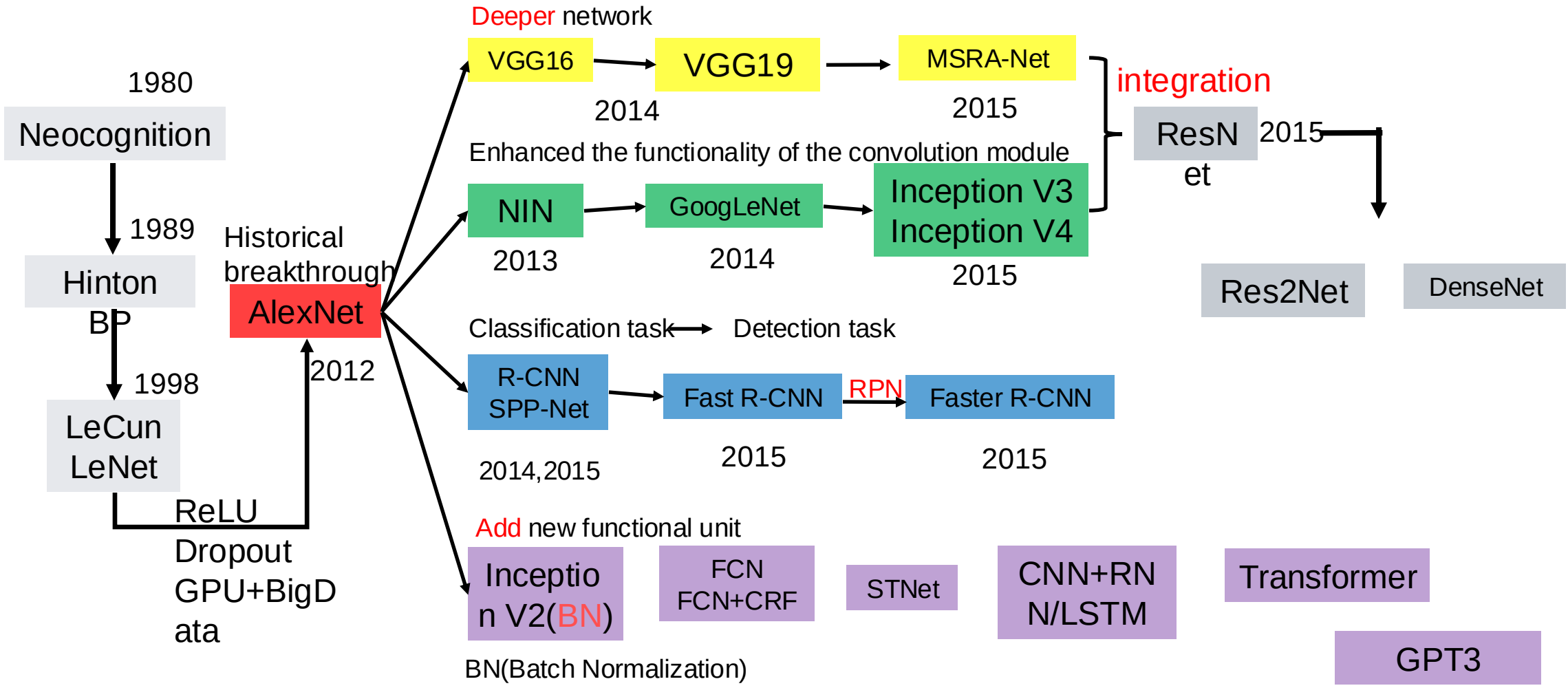
Training stage :

- ✓ Forward propagation
  -Taking a sample (X, Yp) from the sample set and put the X into the network;
  -Calculating the corresponding actual output Op.
- ✓ Back propagation
  -Calculating the difference between the actual output Op and the corresponding ideal output Yp;
  -Adjusting the weight matrix by minimizing the error.

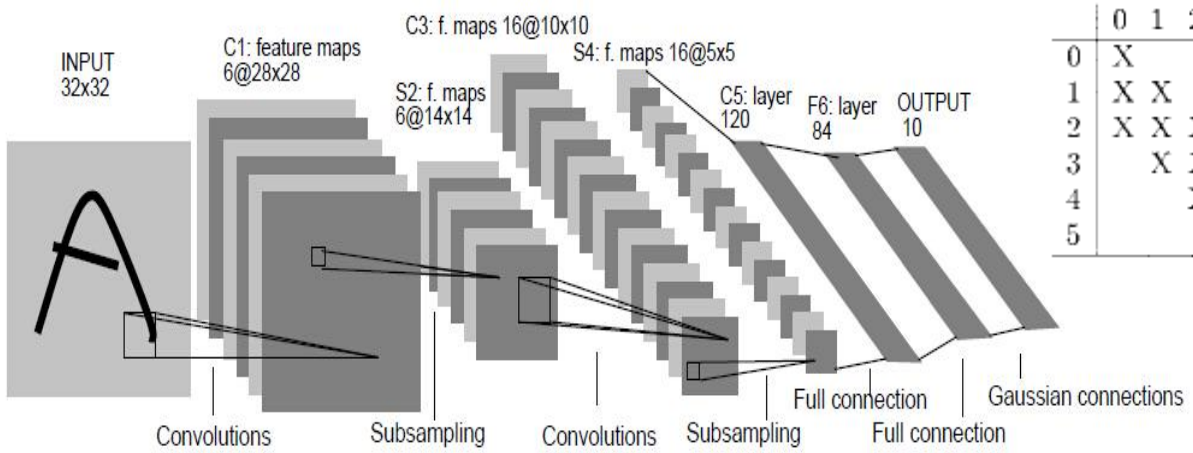
Testing stage :

Putting different images and labels into the trained convolution neural network and comparing the output and the actual value of the sample.

CNN Structure Evolution



LeNet(LeCun,1998)



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	X				X	X	X			X	X	X	X		X	X
1	X	X				X	X	X			X	X	X	X		X
2	X	X	X				X	X	X			X		X	X	X
3		X	X	X			X	X	X	X			X		X	X
4			X	X	X			X	X	X	X		X	X		X
5				X	X	X			X	X	X	X		X	X	X

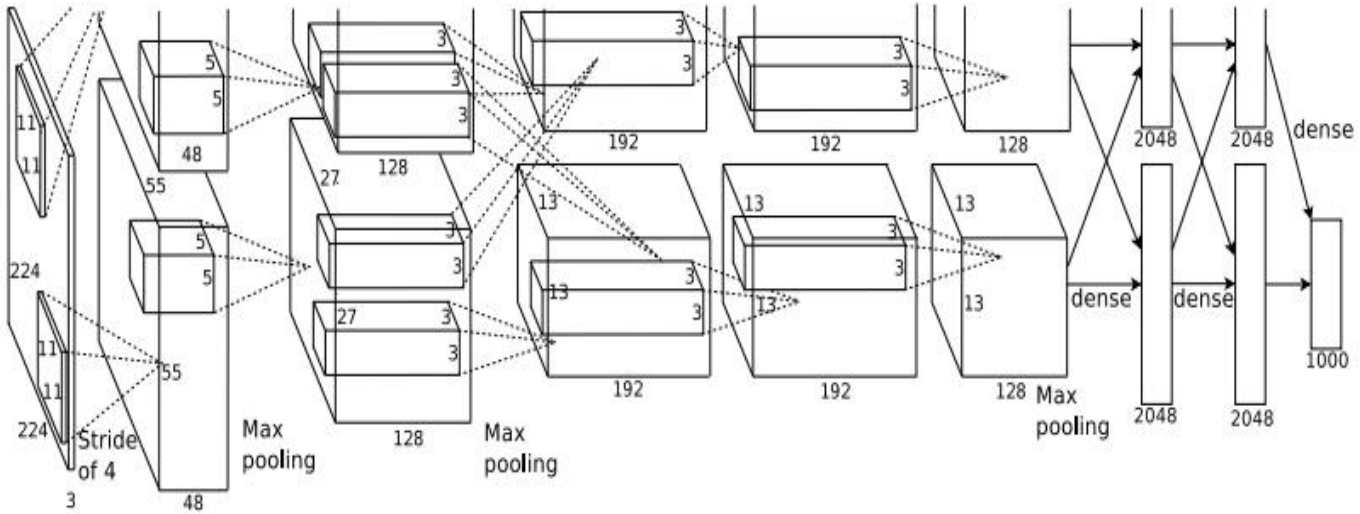
- ✓ LeNet is a convolutional neural network designed by Yann LeCun for handwritten numeral recognition in 1998. It is one of the most representative experimental systems in early convolutional neural networks.
- ✓ LeNet includes the convolution layer, pooling layer and full-connected layer, which are the basic components of modern CNN network. LeNet is considered to be the beginning of the CNN.

network structure :

3 convolution layers + 2 pooling layers+1 fully connected layer + 1 output layer

Y. Lecun, L.Bottou, Y. Bengio, P. Haffner. GradientBased Learning Applied to Document Recognition. Proceedings of IEEE.

AlexNet(Alex,2012)



Network structure : 5 convolution layers + 3 fully connected layers

- ✓ The nonlinear activation function : ReLU(Rectified linear unit)
- ✓ Methods to prevent overfitting : Dropout, Data Augmentation
- ✓ Big Data Training : ImageNet-- image database of million orders of magnitude
- ✓ Others : GPU, LRN(local response normalization) layer

Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks. NIPS. 2012:1097-1105.

VGG-Net(Oxford University,2014)

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 1: ConvNet configurations (shown in columns). The convolutional layer parameters are denoted as “conv<receptive field size>-<number of channels>”

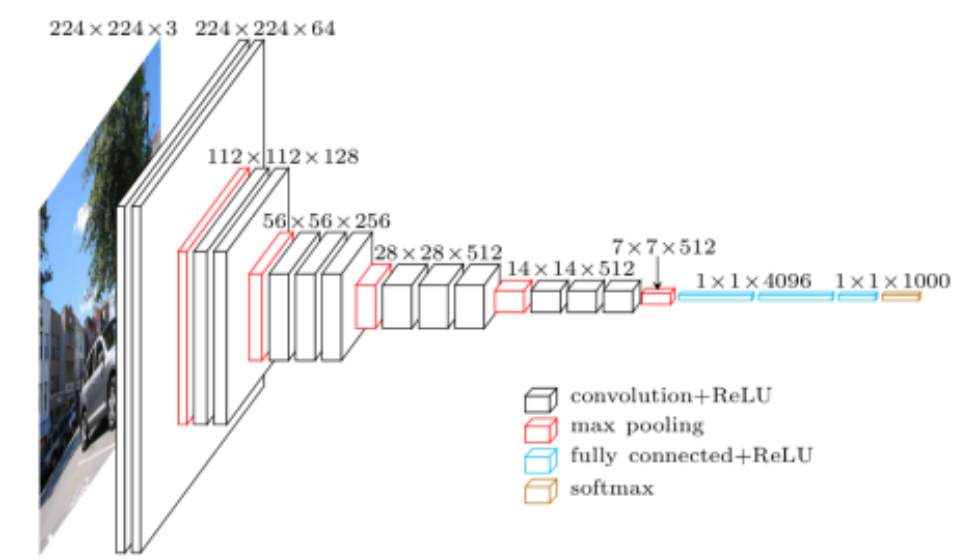


Fig1. Architecture of VGG16

*input : a fixed-size 224*224 RGB image*

*filters : a very small receptive field--3*3,with stride 1*

*Max-pooling : 2*2 pixel window, with stride 2*

Why 3*3 filters?

- ✓ Stacked conv. layers have a large receptive field
- ✓ More non-linearity
- ✓ Less parameters to learn

Simonyan K, Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition[J]. Computer Science, 2014.

GoogLeNet(Inception V1,2014)

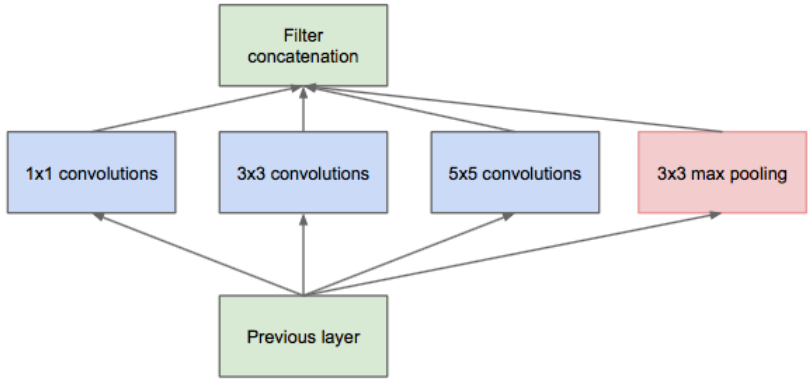


Fig1. Inception module, naïve version

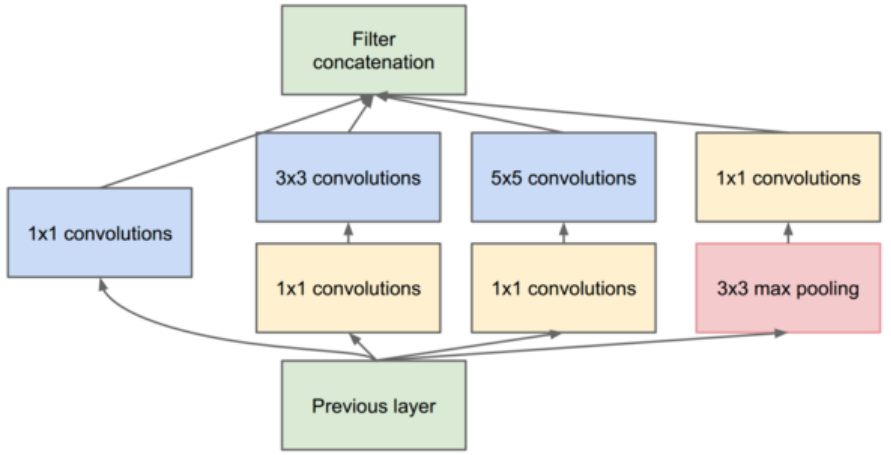


Fig2. Inception module with dimension reductions

- Proposed inception architecture and optimized it
- Canceled the fully connnected layer
- Used auxiliary classifiers to accelerate network convergence

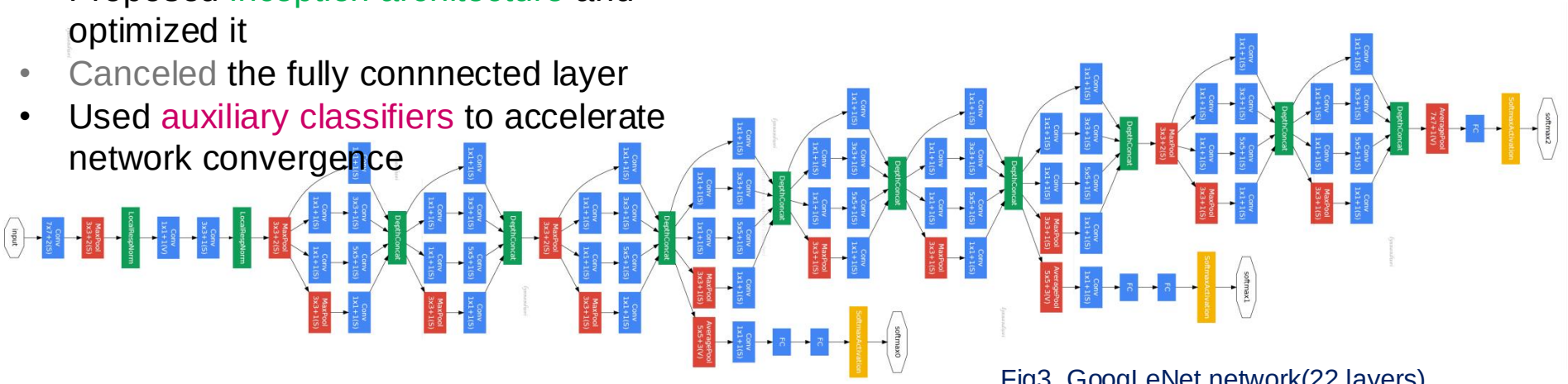


Fig3. GoogLeNet network(22 layers)

Inception:开端，起源。把某物带到这个世界里。盗梦空间

Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2015: 1-9.

ResNet(Kaiwen He, 2015)

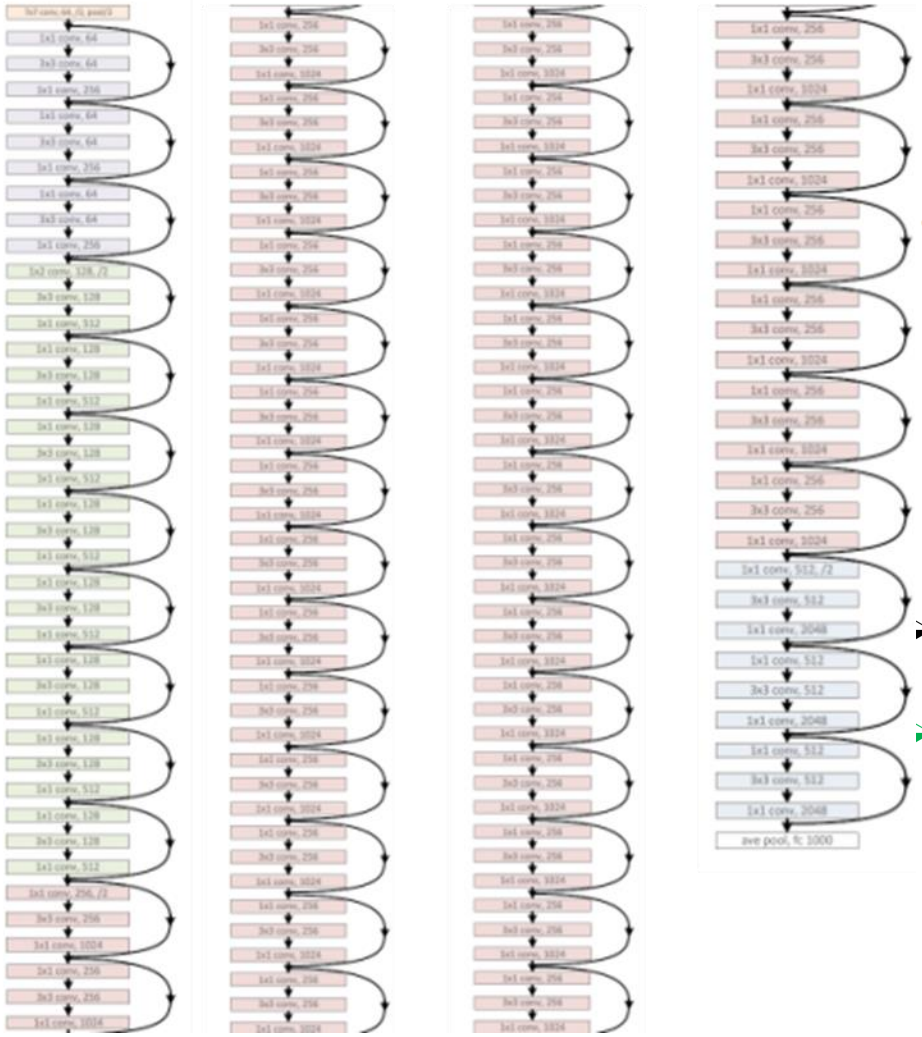


Fig2. ResNet structure(152 layers)

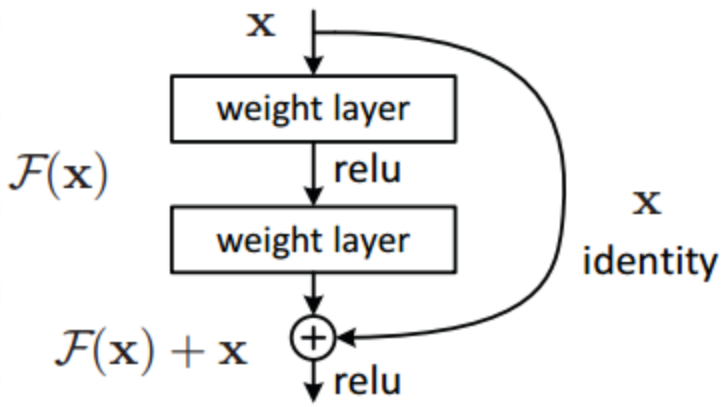
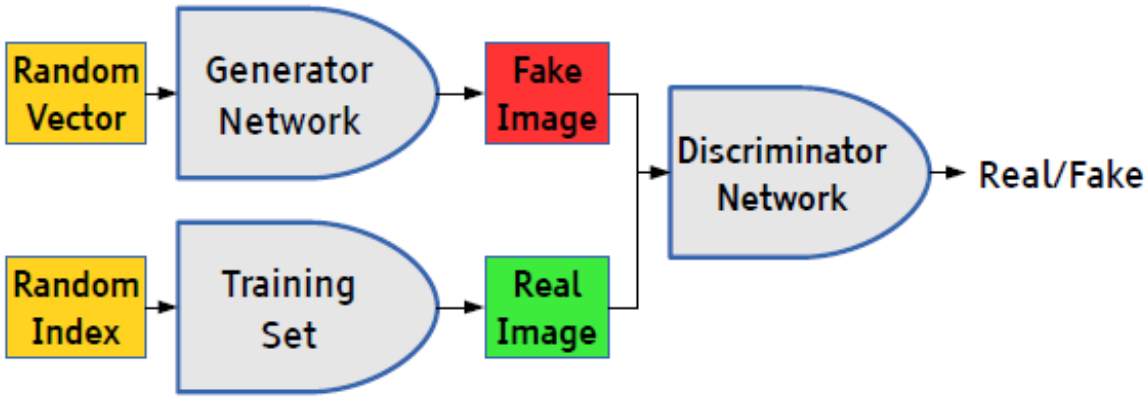


Fig1. Shortcut connections

- A simple and clean framework of training "very" deep networks.
- State-of-the-art performance for
  - Image classification
  - Object detection
  - Semantic Segmentation
  - and more

He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[J]. 2015:770-778.

GANs(Generative Adversarial Networks,2014)



GANs Inspired by zero-sum Game in Game Theory, which consists of a pair of networks - a generator network and a discriminator network.

- The generator network generates a sample from the random vector, the discriminator network discriminates whether a given sample is natural or counterfeit.
- Both networks train together to improve their performance until they reach a point where counterfeit and real samples can not be distinguished.

Applacations:

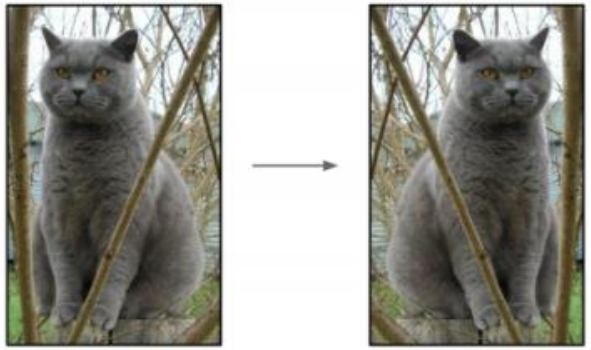
- Image editing
- Image to image translation
- Generate text
- Generate images based on text
- Combined with reinforcement learning
- And more…

Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets[C]//Advances in neural information processing systems. 2014: 2672-2680.

Training tricks

- Data Augmentation
- Dropout
- ReLU
- Batch Normalization

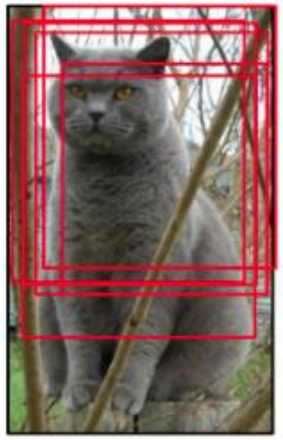
Data Augmentation



Flip horizontally

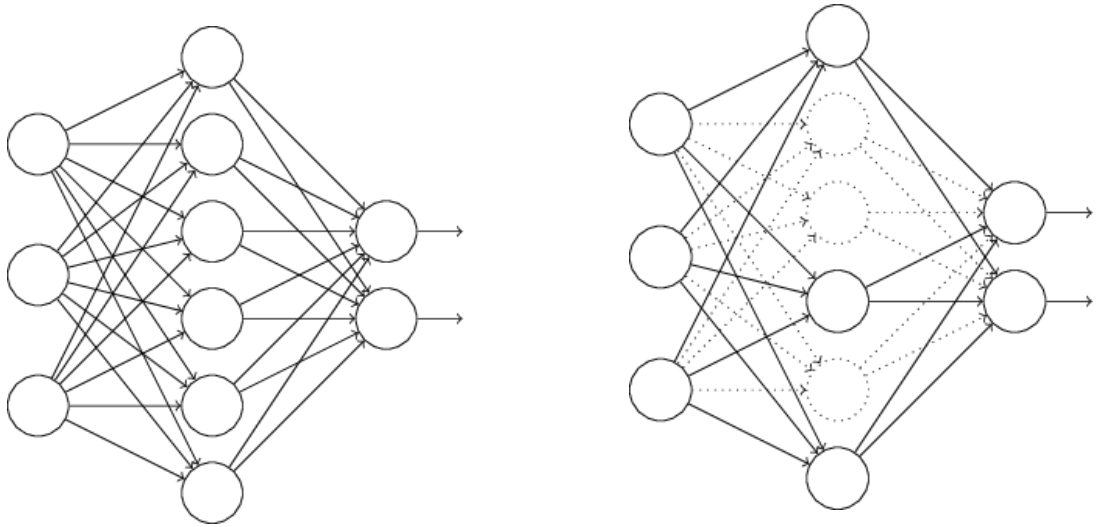
-rotation
-flip
-zoom
-shift
-scale
-contrast
-noise disturbance
-color
-...

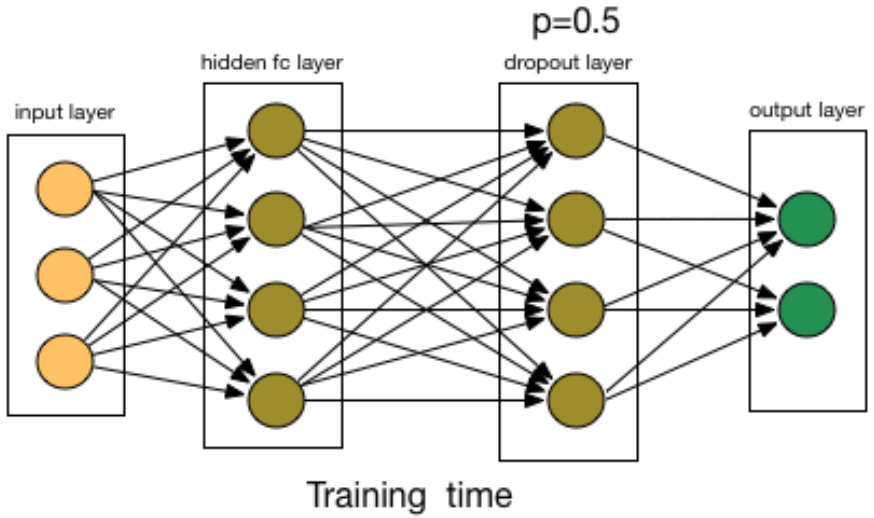
Color jittering



Random crops/scales

Dropout(2012)





- ✓ Dropout consists of setting to zero the output of each hidden neuron with probability p.
- ✓ The neurons which are “dropped out” in this way do not contribute to the forward backpropagation and do not participate in backpropagation.

ReLU(Rectified Linear Unit)

◆ Saturating nonlinearly:

Sigmoid : $f(x) = \frac{1}{1+e^{-x}}$

Hyperbolic tangent :

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

◆ Non-Saturating nonlinearly:

ReLU : $f(x) = \max(0, x) = \begin{cases} 0, x < 0 \\ x, x \geq 0 \end{cases}$

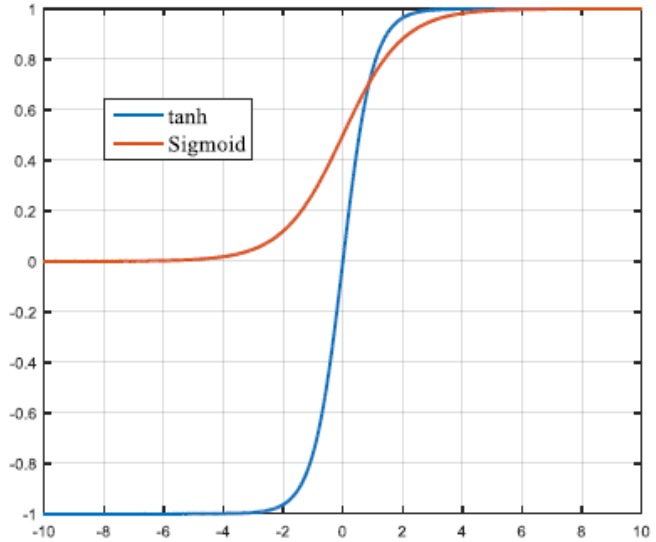
$$f'(x) = \begin{cases} 0, x < 0 \\ 1, x \geq 0 \end{cases}$$

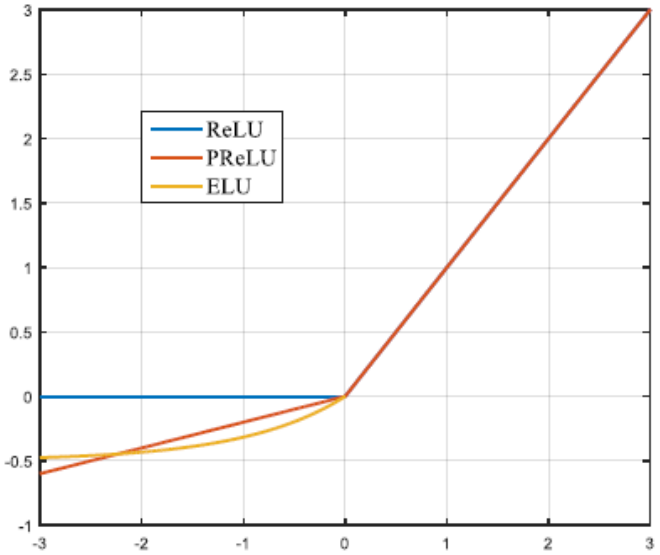
advantages

- ✓ Simplified calculation
- ✓ Avoided gradient disappeared

rectified

-Leaky-ReLU
-Parametric ReLU
-Randomized ReLU





Batch Normalization(2015)

Internal Covariate Shift

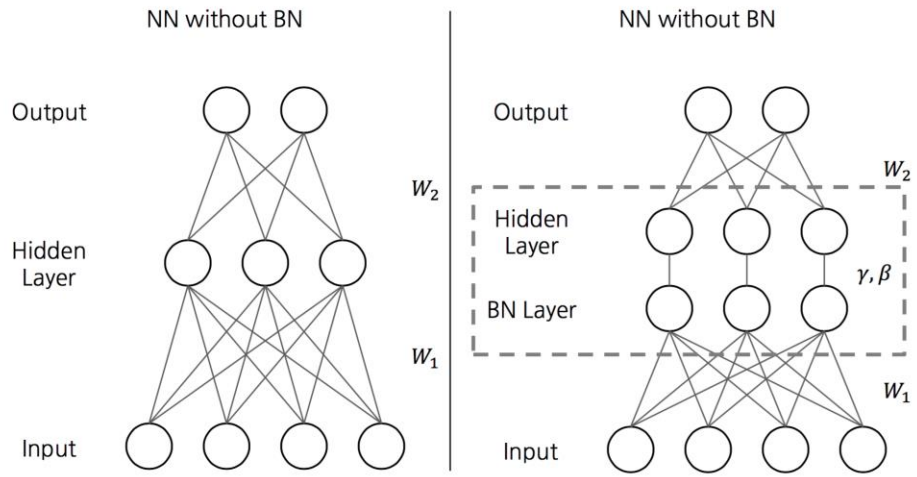
In the input of each layer of the network, insert a normalized layer. For a layer with d-dimensional input x = ($x^{(1)}$ . . . $x^{(d)}$), we will normalize each dimension:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{Var[x^{(k)}]}}$$

$$y^{(k)} = \gamma^{(k)} x^{(k)} + \beta^{(k)}$$

$$\gamma^{(k)} = \sqrt{Var[x^{(k)}]}$$

$$\beta^{(k)} = E[x^{(k)}]$$



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[J]. arXiv preprint

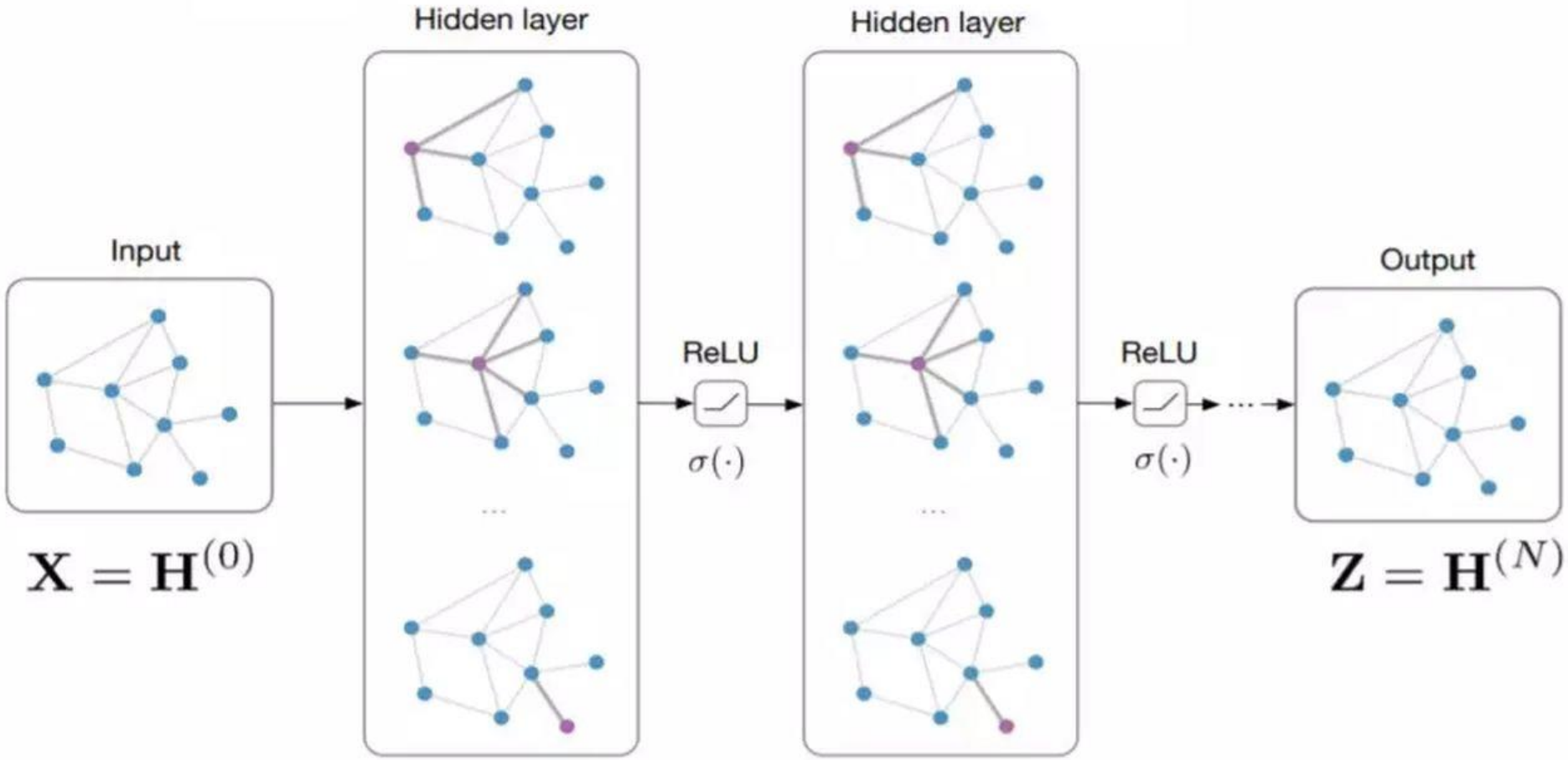
YOLO

图卷积神经网络GCN/GNN

- 深度神经网络本质上可以分成两部分
  - 特征构建部分（将节点间关联变换为节点特征）
  - 分类部分
- 关于特征构建
  - 图像、序列（文本）取得了重大成功
  - 在网络数据上（如社交网络）将节点给特征化可将深度学习推广到更广阔的应用领域上。
  - 互联网推荐、生物信息网络
  - Graph Embedding、Network Embedding、Network Repr. Learning

图卷积的示意

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



$$\mathbf{H}^{(l+1)} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

AdaGrad优化算法

学习率能够随着更新次数的增加逐渐降低

梯度平方累积和的平方根。此项能够累积各个参数gt,i的历史梯度平方，频繁更新的梯度，则累积的分母项逐渐偏大，那么更新的步长(stepsize)相对就会变小，而稀疏的梯度，则导致累积的分母项中对应值比较小，那么更新的步长则相对比较大。

$$g_t \leftarrow +\frac{1}{m}\nabla_\theta \sum_i L(f(x_i;w),y_i)$$

计算梯度

$$r \leftarrow r + g_t^2$$

累积平方梯度

$$\Delta w = -\frac{\eta}{\varepsilon + \sqrt{r}} g_t$$

计算更新 $g_t = \nabla_\theta J(\theta_{t-1})$

$$w \leftarrow w + \Delta w$$

应用更新 $\theta_{t+1} = \theta_t - \alpha \cdot g_t / \sqrt{\sum_{i=1}^{t} g_t^2}$.

g_t为第t次的梯度，r为梯度累积变量，r的初始值为0，会一直递增。η为全局学习率，需要手动设置。ε为小常数，为了数值稳定大约设置为10^{-7}。

John Duchi(UCB), Elad Hazan(IBM Almaden), Yoram Singer(Google Research). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. 2011.

Model Zoo
模型库

Mind Studio
开发工具

MindX SDK
行业SDK

MindX DL
深度学习使能

MindX Edge
智能边缘使能

软件

[M]S 昇思 MindSpore

深度计算平台框架

MindSpore Extend
GNN/大语言模型/强化学习/微分方程/.....

Mind Data
数据加载
格式转换
数据增强

Mind Insight
调试调优

Mind Armour
密态AI、模型混淆、端云协同隐私保护

MindExpress
NN | Data | 算子
训练 | 推理

仓颉/Julia/3rd前端

MindCompiler
类型推导 | 自动微分 | 自动并行 | 二阶优化
内存优化 | 图算融合 | 流水线执行
量化/剪枝/....
MindIR (Intermediate Representation)
MindAKG(poly自动优化)

MindRT
分布式异构并行 | 算子

MindRT

MindRT Lite/Macro
异构并行 | 算子

深度计算平台

[M]S 昇思 MindSpore

TensorFlow

PyTorch

驱动软件

CANN

CANN：Computing Architecture for Neural Network
统一异构计算架构

NVIDIA CUDA

硬件

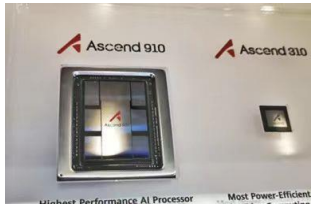
载体(云边端)

端

边

云

芯片、板卡



ADAM:自适应性矩估计(adaptive moment estimation)

- 2015 年由 OpenAI 的 Diederik Kingma 和多伦多大学的 Jimmy Ba 在在 ICLR(Learning Representations)上发表
- 替代传统梯度下降的一阶优化算法，使用一阶矩（均值）和二阶矩（方差）估计。计算更合适的步长
- 适用于非稳目标，可解决高噪和稀疏梯度问题
- 梯度对角缩放的不变性

Diederik Kingma, Jimmy Ba. Adam: A method for Stochastic Optimization, that the name is derived from adaptive moment estimation. ICLR 2015

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1^{st} moment vector)
$v_0 \leftarrow 0$ (Initialize 2^{nd} moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

MindSpore

全场景统一API
Unified APIs for all scenarios

自动微分 Automatic differentiation	自动并行 Automatic parallelization	自动调优 Automatic tuning

MindSpore IR 计算图表达
MindSpore intermediate representation (IR) for computation graphs

On-Device执行 On-device execution	Pipeline并行 Pipeline parallelism	深度图优化 Deep graph optimization

端-边-云按需协作分布式架构（部署、调度、通信等）
Device-edge-cloud cooperative distributed architecture (for deployment, scheduling, communications, etc.)

MindX SDK 行业SDK

MindX DL 深度学习使能

MindX Edge 智能边缘使能

MindSpore Extend
大语言模型/强化学习/微分方程/.....

MindExpress
NN 训练 | Data 推理 | 算子

仓颉/Julia/3rd前端

MindCompiler
自动微分 | 自动并行 | 二阶优化 | 图算融合 | 流水线执行 | 量化/剪枝/....
MindIR (Intermediate Representation)
MindAKG(poly自动优化)

MindRT
异构并行 | 算子

MindRT Lite/Macro
异构并行 | 算子

软

处理器：Ascend、GPU、CPU

深度计算平台

昇思 MindSpore

TensorFlow

PyTorch

驱动软件

CANN

CANN：Computing Architecture for Neural Network
统一异构计算架构

NVIDIA CUDA

硬件

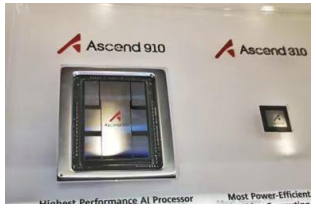
载体(云边端)

端

边

云

芯片、板卡



华为深度学习框架：昇腾架构

行业应用

能源、金融、公共、交通、运营商、制造、教育等更多行业应用

应用使能

ModelArts　HiAi Service　第三方平台

MindX 使能应用

MindX DL
深度学习使能

MindX Edge
智能边缘使能

ModelZoo
优选模型库

MindX SDK
行业SDK

AI框架

MindSpore

匹配昇腾AI处理器算力的全场景深度学习框架

AI异构计算框架

CANN

统一异构计算架构，释放昇腾硬件澎湃算力

Atlas系列硬件

端　边　云

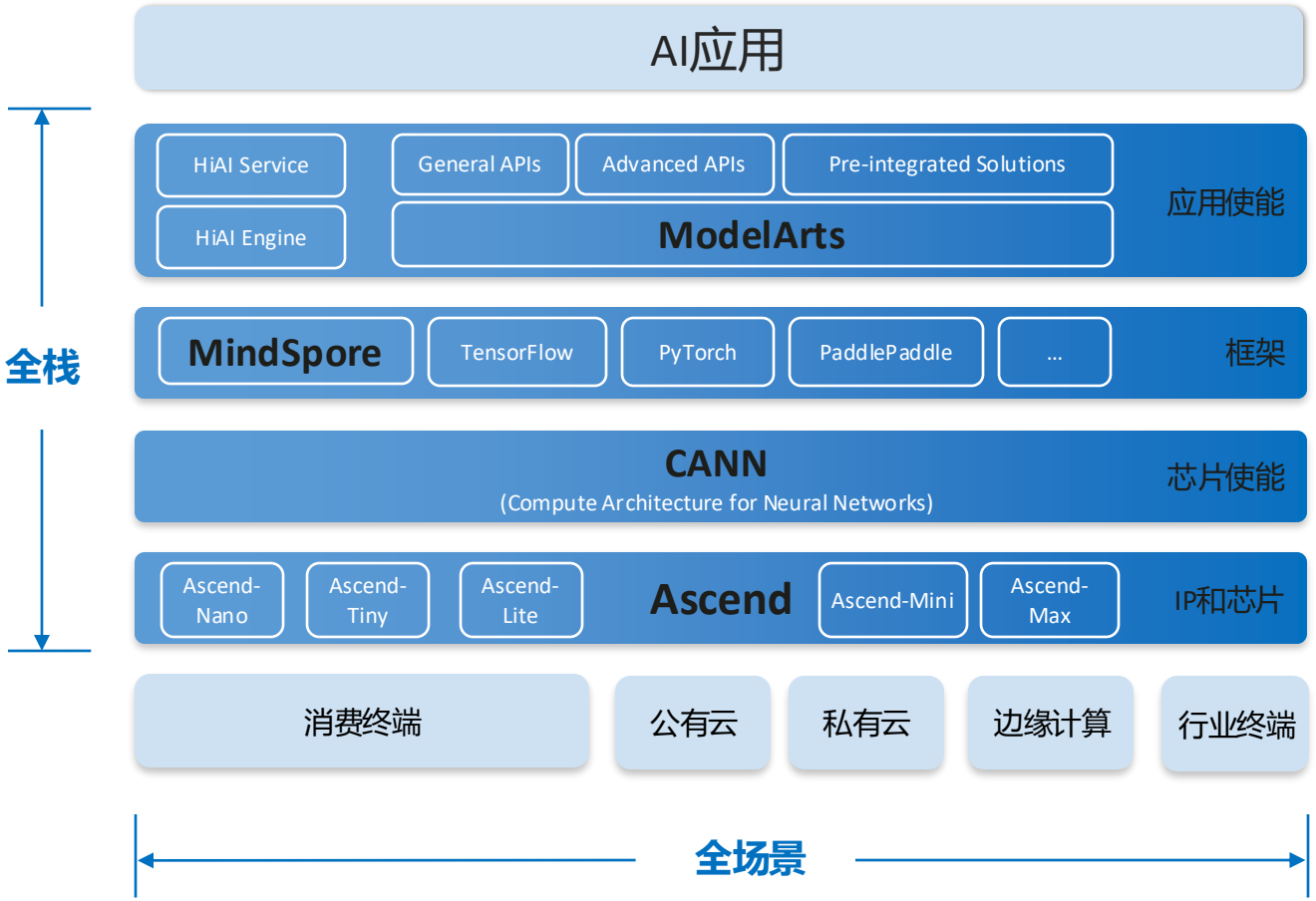
Cloud

全流程开发工具链 MindStudio

管理运维工具 FusionDirector/Smart

Ascend 昇腾社区 ascend.huawei.com

华为全栈AI解决方案



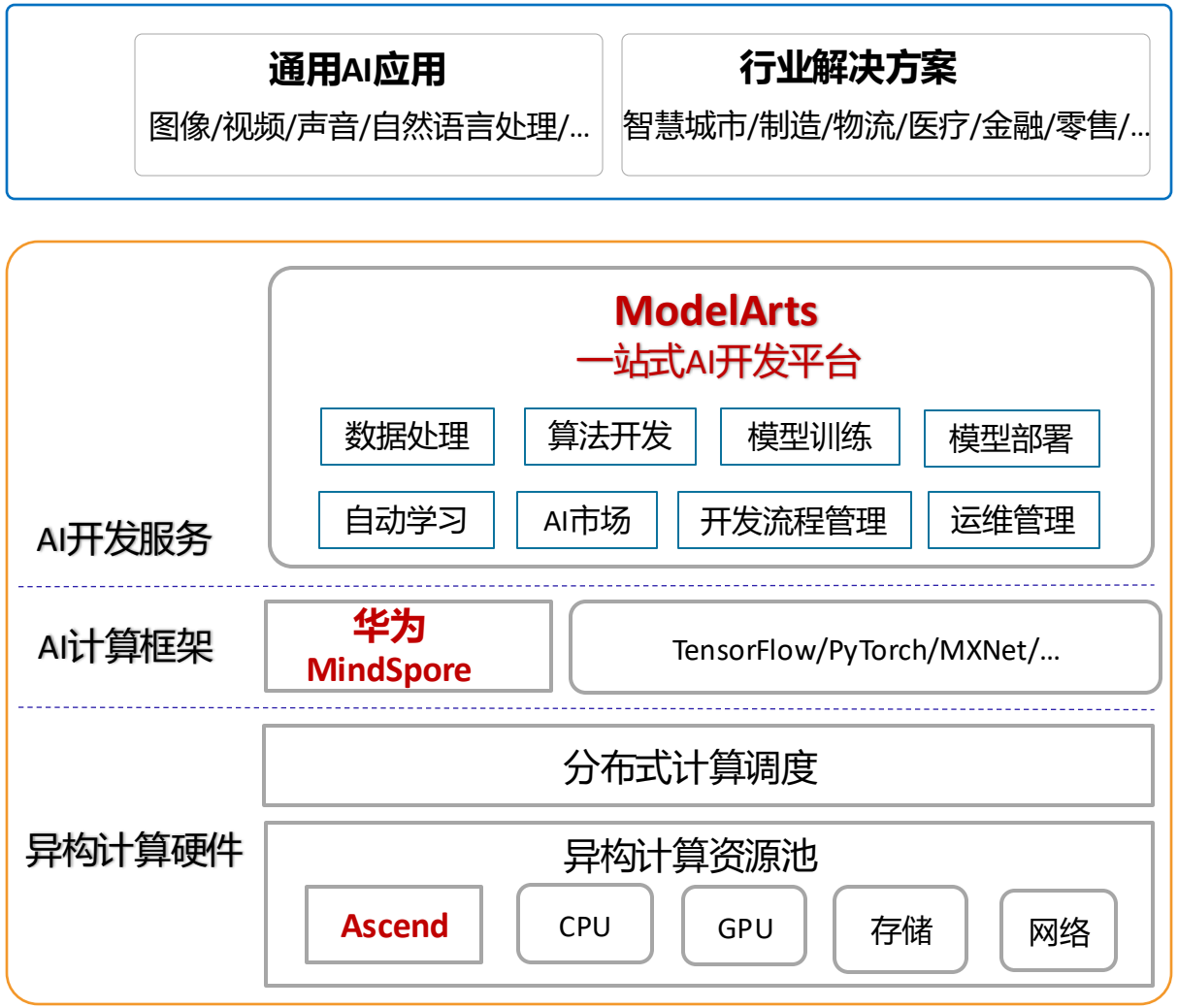
应用使能：提供全流程服务（**ModelArts**），分层API和预集成方案

MindSpore：支持端、边、云独立和协同的训练和推理框架

CANN：芯片算子库和高度自动化算子开发工具

Ascend：基于统一、可扩展架构的系列化AI IP 和芯片

华为昇腾AI解决方案



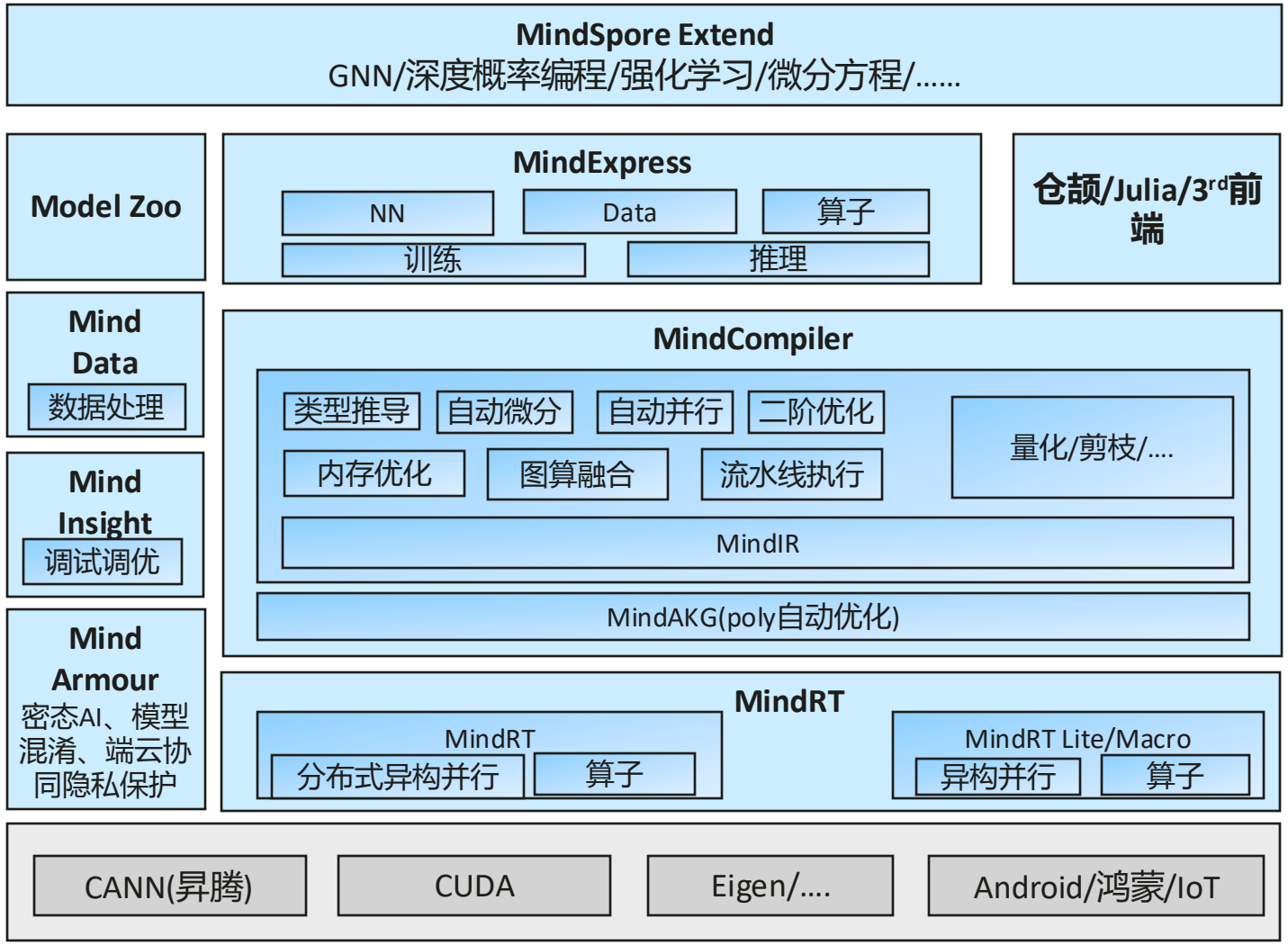
华为面向各行业AI应用的开发与研究，提供全流程、普惠的基础平台类服务，支持多种类别的**通用AI能力**（包括但不限于图像、视频、语音、自然语言处理，对话机器人等），能够**帮助企业应用开发者迅速集成AI能力到业务应用**，也可以**支持城市、制造等行业解决方案**实现。

异构计算硬件是加速AI计算的异构计算资源池，包括高性能的AI计算芯片使能的服务器（GPU，华为Ascend），高速高性能网络和存储，作为整体平台的硬件基础。

AI计算框架MindSpore支持端、边、云独立和协同的统一的AI领域的训练和推理场景，支持不同的资源部署环境，以统一分布式架构支持机器学习和深度学习，提供跨平台、大规模、高并发的AI算法运行软件环境。

AI开发服务（ModelArts）提供全流程的AI开发服务，海量数据处理、大规模分布式训练、自动化模型生成，端-边-云模型按需部署，运维管理，帮助用户快速创建和部署模型、管理全周期AI工作流，满足不同开发层次的需要，降低AI开发和使用门槛，实现系统的平滑、稳定、可靠运行。

华为MindSpore逻辑架构



设计目标

① **Beyond AI**：NN应用□通用AI+数值计算
- 关键特性：通用的张量可微计算

② **分布式并行原生**：支撑AI模型突破万亿参数
- 关键特性：自动并行、内存约束编程、二阶优化

③ **图算深度融合**：充分发挥AI芯片的算力
- 关键特性：图算联合优化、Poly自动优化

④ **全场景企业级能力**：实现灵活部署和协同，安全可信、可解释
- 关键特性：极轻量运行时、私密训练、自适应模型生成、量化训练、可解释AI

设计理念：AI的“JDK”

① **表达/优化/运行解耦**：实现多前端、跨芯片、跨平台

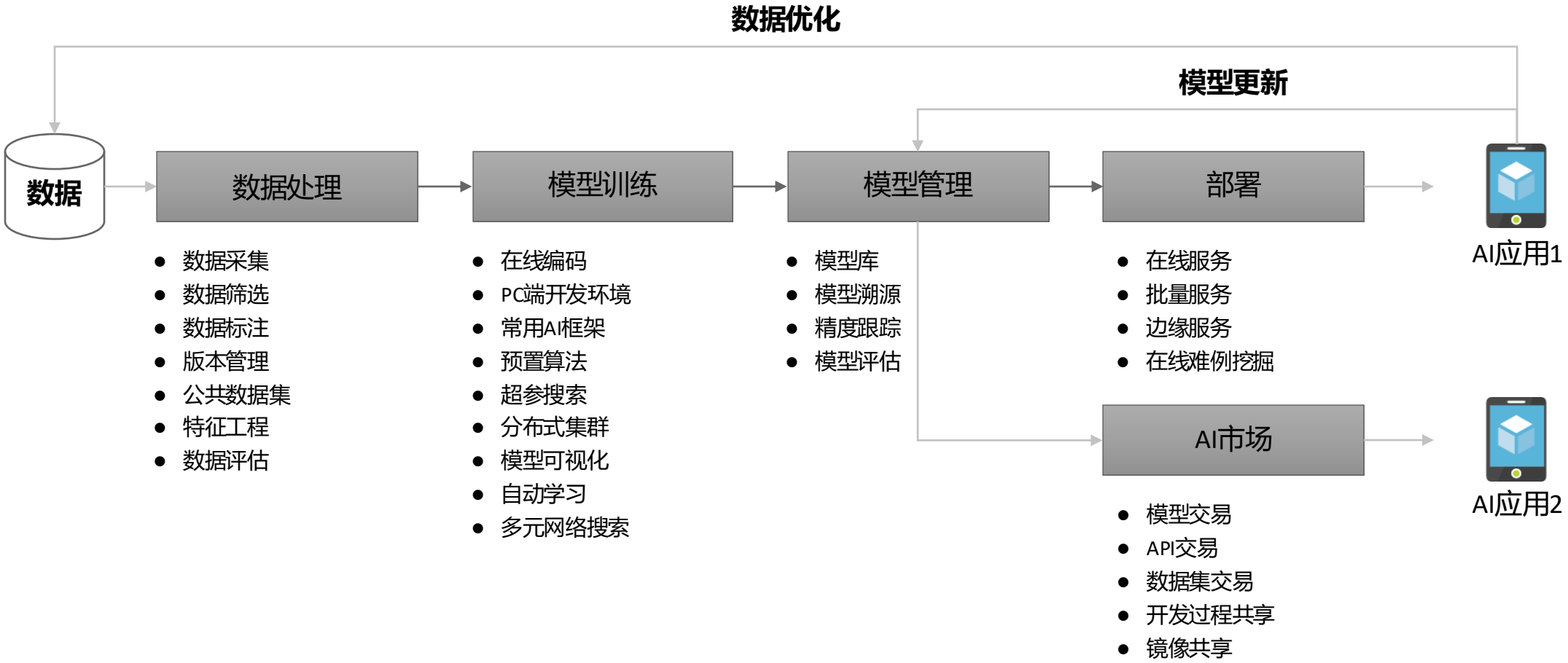
② **开放**：开放通用图编译/运行能力给三方框架

③ **全场景统一架构**：接口和IR统一，AI应用可平滑流动

机器学习应用开发流程

面向不同AI开发者：

- 应用开发者
- 公民数据科学家
- AI专家
- 智能运维（AI Ops）

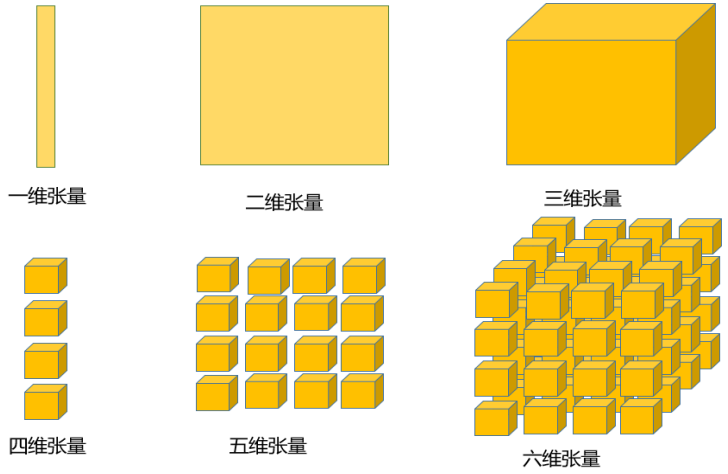


MindSpore开发环境准备

- 本机环境
  - Miniconda环境安装
  - 使用conda创建虚拟环境
  - 使用pip安装各种包（需要更换成国内源）
  - 安装MindSpore
  - 配置jupyter notebook或PyCharm
- 华为云
  - 注册华为云账号
  - 获取华为云优惠券或充值
  - 打开ModelArts的Notebook，开始进行编程

编程概念：张量（Tensor）

- MindSpore中最基础的数据结构是张量 (Tensor)，是计算的基础：
  - 是Parameter（权重和偏置）的载体，Feature Map的载体；
  - 可与numpy.ndarray无缝转换。



1. Tensor声明和使用

```
t1 = Tensor(np.zeros([1, 2, 3]),
    ms.float32)
assert isinstance(t1, Tensor)
assert t1.shape == (1, 2, 3)
assert t1.dtype == ms.float32
```

2. Tensor是Parameter的数据载体

Parameter的属性：

- default_input: Tensor
- name: str
- requires_grad: bool
- layerwise_parallel: bool

3. Tensor常见的操作

```
// 转换成Numpy数组
asnumpy()
// 获取Tensor的大小
size()
// 获取Tensor的维数
dim()
// 获取Tensor的数据类型
dtype
// 设置Tensor的数据类型
set_dtype()
// 获取Tensor的形状.
shape
```

编程概念：算子（Operation）

常用的Operation：

- array: Array相关的算子
  - ExpandDims - Squeeze
  - Concat - OnesLike
  - Select - StridedSlice
  - ScatterNd
  - ……
- math: 数学计算相关的算子
  - AddN - Cos
  - Sub - Sin
  - Mul - LogicalAnd
  - MatMul - LogicalNot
  - RealDiv - Less
  - ReduceMean - Greater …..
- nn: 网络类算子
  - Conv2d - MaxPool
  - Flatten - AvgPool
  - Softmax - TopK
  - ReLU - SoftmaxCrossEntropy
  - Sigmoid - SmoothL1Loss
  - Pooling - SGD

```
>>> data1 = Tensor(np.array([[0, 1], [2, 1]]).astype(np.int32))
>>> data2 = Tensor(np.array([[0, 1], [2, 1]]).astype(np.int32))
>>> op = P.Concat()
>>> output = op((data1, data2))
```

```
>>> input_x = Tensor(np.array([0.24, 0.83, 0.31, 0.09]),
mindspore.float32)
>>> output = cos(input_x)
```

```
>>> input_x = Tensor(np.array([-1, 2, -3, 2, -1]),
mindspore.float16)
>>> relu = nn.ReLU()
>>> relu(input_x)
[0.  2.  0.  2.  0.]
```

编程概念：Cell

- Cell是MindSpore核心编程结构，定义了执行计算的基本模块。Cell对象有以下成员方法：
  - __init__，初始化参数（Parameter），子模块（Cell），算子（Primitive）等组件，进行初始化的校验；
  - construct，定义执行的过程，有一些语法限制。图模式时，会被编译成图来执行；
  - bprop（可选），自定义模块的反向。未定义时，框架会自动生成反向图，计算construct的反向。
- 预定义的Cell
  - nn算子：ReLU, Dense, Conv2d
  - Loss, Optimizer, Metrics
  - Wrapper：TrainOneStepCell, WithLossCell, WithGradCell,

```
class MyNet(nn.Cell):
    def __init__(self, in_channel, out_channel):
        super(MyNet, self).__init__()
        self.fc = nn.Dense(in_channel, out_channel, weight_init='normal',  bias_init='zero', has_bias=True)
        self.relu = nn.ReLU()
    def construct(self, x):
        x = self.fc(x)
        x = self.relu(x)
        return x
```

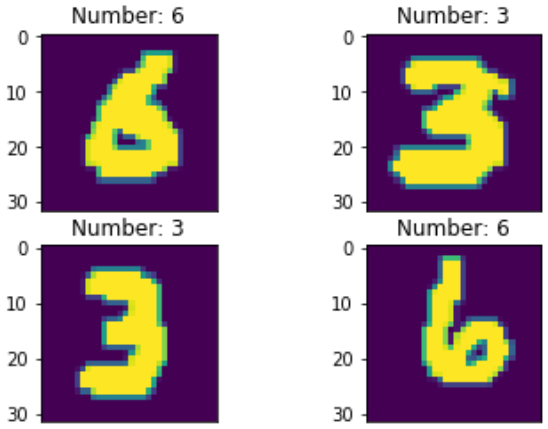
常用模块

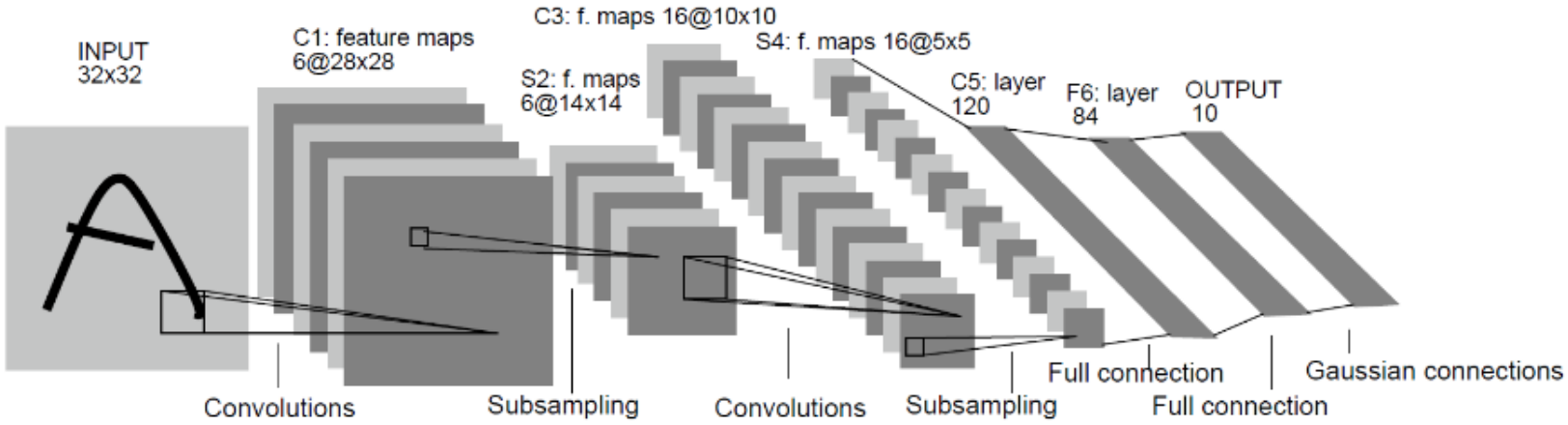
模块	描述
mindspore.dataset	常见Dataset加载和处理接口，如MNIST, CIFAR-10, VOC, COCO, ImageFolder, CelebA等； transforms提供基于OpenCV、PIL及原生实现的数据处理、数据增强接口； text提供文本处理接口；
mindspore.common	Tensor（张量），Parameter（权重、偏置等参数），dtype （数据类型）及Initializer（Parameter初始化）等接口
mindspore.context	设置上下文，如设置Graph和PyNative模式、设备类型、中间图或数据保存、Profiling、设备内存、自动并行等
mindspore.nn	Neural Network常用算子，如Layer（层）、Loss（损失函数）、Optimizer（优化器）、Metrics（验证指标）以及基类Cell和Wrapper
mindspore.ops	原语算子operations（需初始化），组合算子composite，功能算子（已初始化的原语算子）
mindspore.train	Model（训练、验证、推理接口）、callback（损失监控、Checkpoint保存、Summary记录等），serialization（加载Checkpoint、导出模型等），quant（量化）

API文档：https://www.mindspore.cn/api/zh-CN/master/index.html

基于LeNet-5的手写数字识别

- MNIST是一个手写数字数据集，训练集包含60000张手写数字，测试集包含10000张手写数字，共10类。
- 卷积神经网络（Convolutional Neural Networks, CNN）的研究始于二十世纪80至90年代，LeNet是最早出现的卷积神经网络之一。
- 在LeNet的基础上，1998年Yann LeCun等构建了卷积神经网络LeNet-5并在手写数字的识别问题中取得成功 。LeNet-5及其后产生的变体定义了现代卷积神经网络的基本结构，可谓入门级神经网络模型。

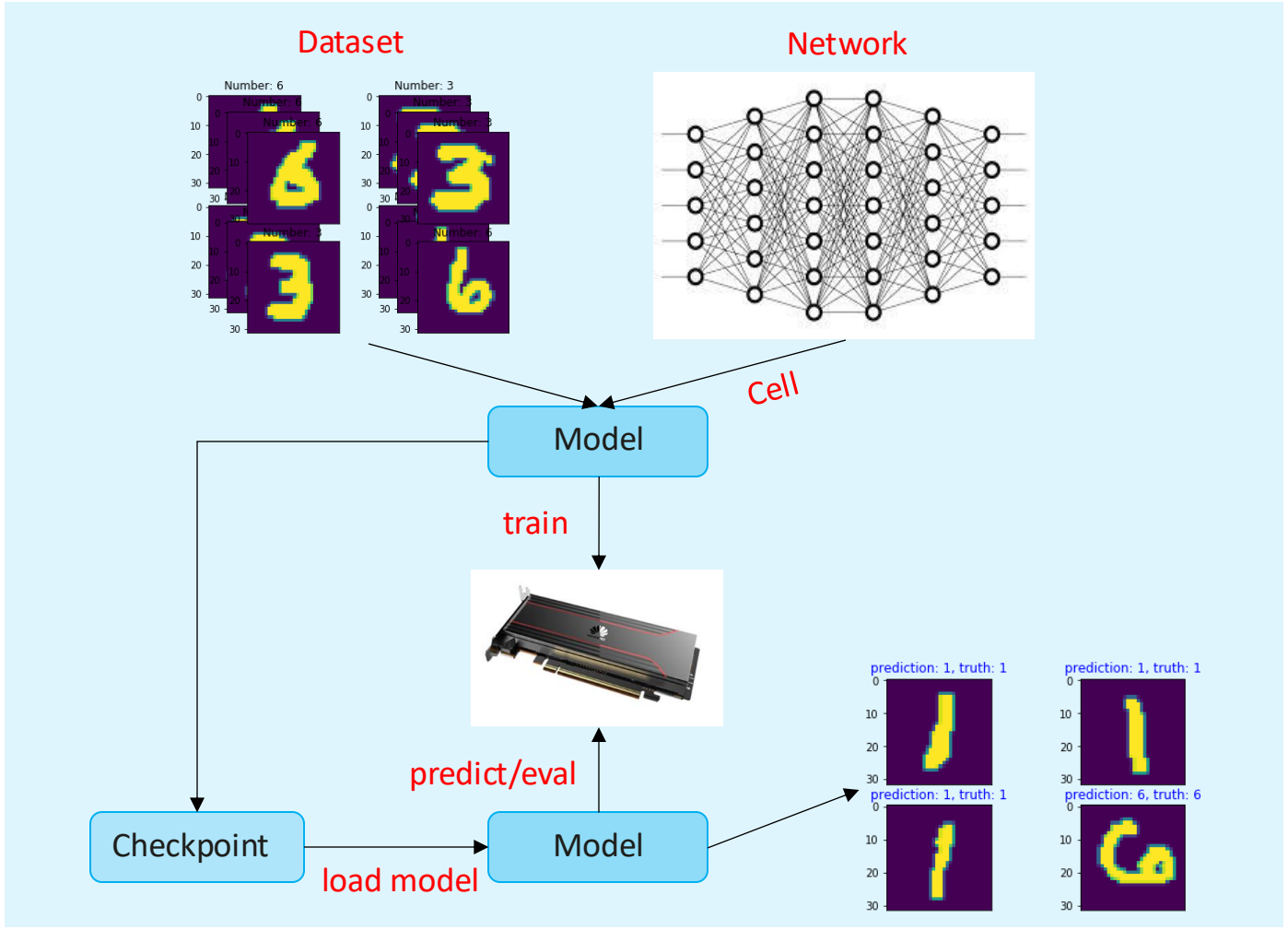




MNIST官网：http://yann.lecun.com/exdb/mnist/
LeNet-5论文：http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

开发流程

- 数据处理
  - 数据加载
  - 数据增强
- 模型定义
  - 定义网络
  - 损失函数
  - 优化器
- 模型训练
  - Loss监控
  - 验证
  - 保存Checkpoint
- 模型推理
  - 推理
  - 部署



代码实例

```
import os

import mindspore as ms
import mindspore.context as context
import mindspore.dataset.transforms.c_transforms as C
import mindspore.dataset.transforms.vision.c_transforms as CV

from mindspore import nn
from mindspore.train import Model
from mindspore.train.callback import LossMonitor
```

① 导入标准库、第三方库和MindSpore模块。

```
context.set_context(mode=context.GRAPH_MODE, device_target='Ascend')
```

② 设置上下文：GRAPH模式、Ascend设备。

```
def create_dataset(data_dir, training=True, batch_size=32, resize=(32, 32), rescale=1/(255*0.3081), shift=-0.1307/0.3081, buffer_size=64):
    data_train = os.path.join(data_dir, 'train')
    data_test = os.path.join(data_dir, 'test')
    ds = ms.dataset.MnistDataset(data_train if training else data_test)

    ds = ds.map(input_columns=["image"], operations=[CV.Resize(resize), CV.Rescale(rescale, shift), CV.HWC2CHW()])
    ds = ds.map(input_columns=["label"], operations=C.TypeCast(ms.int32))
    ds = ds.shuffle(buffer_size=buffer_size).batch(batch_size, drop_remainder=True)

    return ds
```

② 数据处理：数据集加载、缩放、归一化、格式转换、洗牌、批标准化。

```
class LeNet5(nn.Cell):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, stride=1, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, stride=1, pad_mode='valid')
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(400, 120)
        self.fc2 = nn.Dense(120, 84)
        self.fc3 = nn.Dense(84, 10)

    def construct(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)

        return x
```

② 模型定义：算子初始化（参数设置），网络构建。

```
def train(data_dir, lr=0.01, momentum=0.9, num_epochs=3):
    ds_train = create_dataset(data_dir)
    ds_eval = create_dataset(data_dir, training=False)

    net = LeNet5()
    loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')
    opt = nn.Momentum(net.trainable_params(), lr, momentum)
    loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())

    model = Model(net, loss, opt, metrics={'acc', 'loss'})
    model.train(num_epochs, ds_train, callbacks=[loss_cb], dataset_sink_mode=False)
    metrics = model.eval(ds_eval, dataset_sink_mode=False)
    print('Metrics:', metrics)
```

② 模型训练和验证：实例化net, loss, optimizer, loss_monitor，调用Model接口进行训练和验证。

数据处理Pipeline

- 数据是深度学习的基础，高质量的数据输入会在整个深度神经网络中起到积极作用。
- 在训练开始之前，由于数据量有限，或者为了得到更好的结果，通常需要进行数据处理与数据增强，以获得能使网络受益的数据输入。

- 数据经过处理和增强，像流经管道的水一样源源不断的流向训练系统。

加载数据集

Dataset structure	使用简介

ImageFolderDatasetV2	# 读取图目录名作为类别标签，每个目录下的图片为同一类
	# Example: cat/image1.jpg, dog/image2.jpg
	ds = ds.ImageFolderDatasetV2(DATA_DIR, class_indexing={"cat":0,"dog":1})

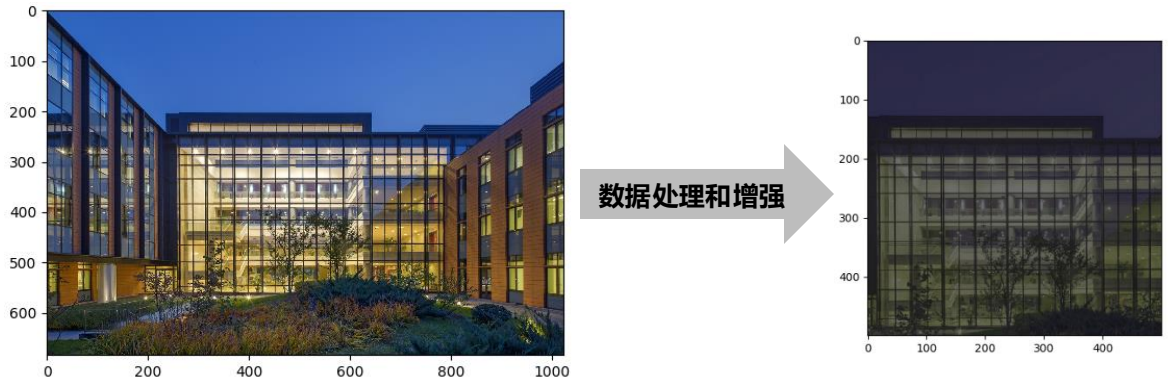
MnistDataset	ds = ms.dataset.MnistDataset(DATA_DIR)
Cifar10Dataset	ds = ms.dataset.Cifar10Dataset(DATA_DIR)
CocoDataset	ds = ds.CocoDataset(DATA_DIR, annotation_file=annotation_file, task='Detection')
MindRecord	ds = ds.MindDataset(FILE_NAME)
GeneratorDataset	ds = ds.GeneratorDataset(GENERATOR, [“image", “label"])

TextFileDataset	dataset_files = ["/path/to/1", "/path/to/2"] # contains 1 or multiple text files
	ds = ds.TextFileDataset(dataset_files=dataset_files)

GraphData	ds = ds.GraphData(DATA_DIR, 2) nodes = ds.get_all_edges(0)

数据处理

- 数据处理
  - 以图片为例：将原始图片进行裁剪、大小缩放、归一化、格式转换等，以便深度学习模型能够使用。
- 数据增强
  - 一种创造 “新” 数据的方法，从有限数据中生成 “更多数据”，防止过拟合现象



数据增强：有限数据和复杂模型→过拟合

格式转换：MindSpore支持channel first

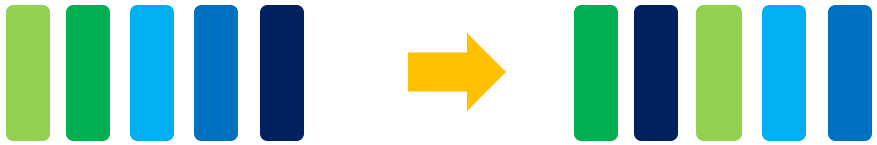

```
operations = [
    CV.Resize(resize),
    CV.Rescale(rescale, shift),
    CV.HWC2CHW()
]
ds = ds.map(input_columns=["image"], operations=operations)
ds = ds.shuffle(buffer_size=buffer_size).batch(batch_size,
drop_remainder=True)
```

常用操作

(1)shuffle

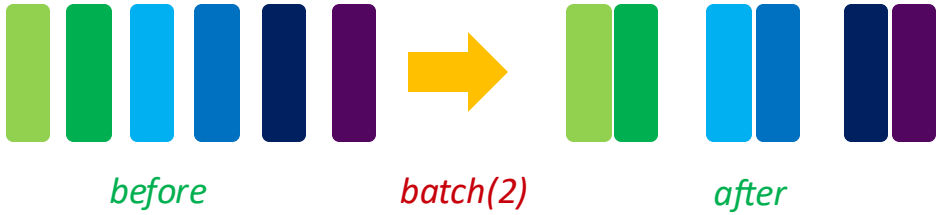
shuffle操作用来打乱数据集中的数据排序。

越大的buffer_size意味着越高的混洗度，但也意味着会花费更多的时间和计算资源。



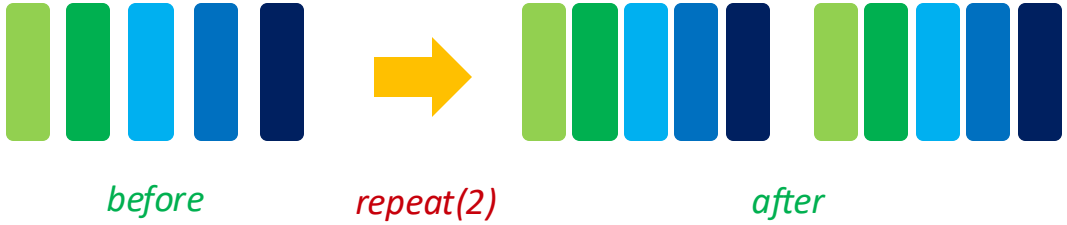
(2)batch

在训练时，数据可能需要分批batch处理。



(3)repeat

可以通过repeat的方式增加训练数据量，通常放在batch操作之后。

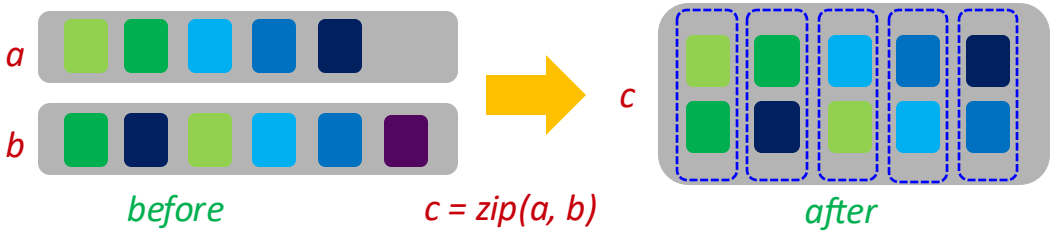


(4)zip

将多个数据集合并成一个。

若两个数据集的列的名字相同，则不能合并。

若两个数据集的行数不同，合并后的行数以较小的为准。



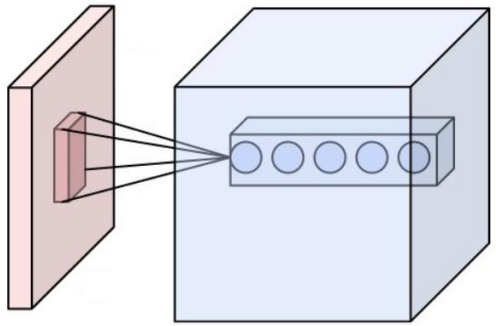
代码实例

```
def create_dataset(data_dir, training=True, batch_size=32, resize=(32, 32),
                   rescale=1/(255*0.3081), shift=-0.1307/0.3081, buffer_size=64):
    # 训练集和测试集路径
    data_train = os.path.join(data_dir, 'train')
    data_test = os.path.join(data_dir, 'test')
    # 加载MNIST数据集
    ds = ms.dataset.MnistDataset(data_train if training else data_test)

    # 通过map方法对每张图片应用数据处理操作：
    # Resize：缩放图片大小
    # Rescale：将每个像素的灰度值由[0, 255]标准化到[-1, 1]
    # HWC2CHW：将张量格式从NHWC转换成NCHW
    ds = ds.map(input_columns=["image"], operations=[CV.Resize(resize), CV.Rescale(rescale, shift), CV.HWC2CHW()])
    # 将每个标签的数据类型转换为int32
    ds = ds.map(input_columns=["label"], operations=C.TypeCast(ms.int32))
    # 当在Ascend环境上使用数据下沉模型时，设置`dataset_sink_mode=True`
    ds = ds.shuffle(buffer_size=buffer_size).batch(batch_size, drop_remainder=True)

    return ds
```

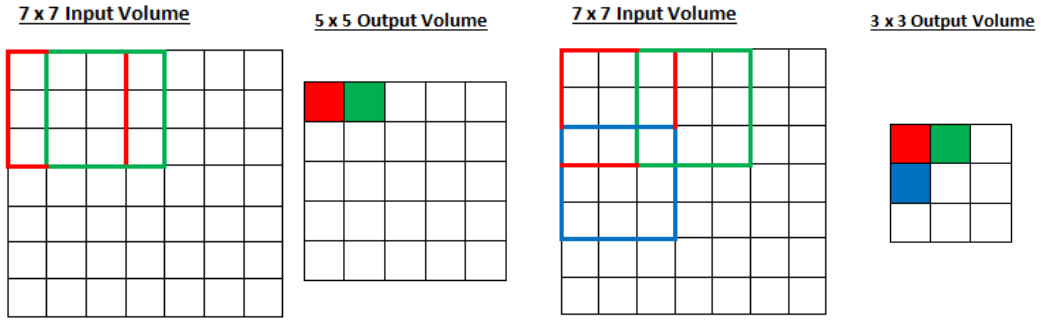
卷积（Convolution）

```
conv1 = nn.Conv2d(in_channels, out_channels,                    ①
        kernel_size=3, stride=1, has_bias=False, weight_init='normal', bias_init='zeros',
        pad_mode='same', padding=0)
```

① Input and Output Channels



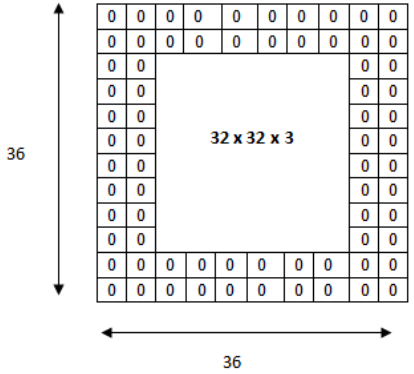
② Kernel Size and Stride



kernel_size=3, stride=1, padding=0

kernel_size=3, stride=1, padding=0

③ Padding



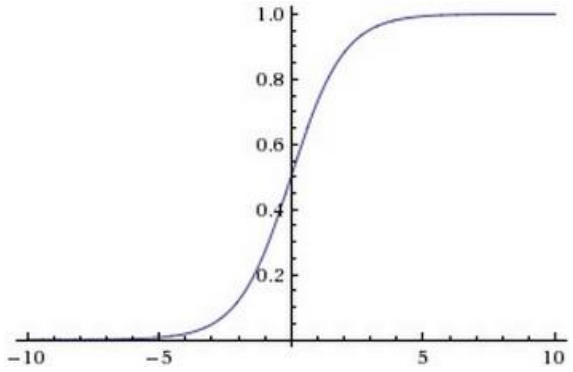
y = conv1(x)

接受一个4维的张量作为输入，返回一个4维的张量作为输出，4维即batch_size x channels x height x width

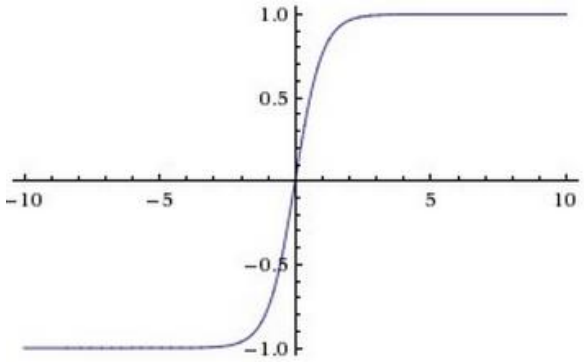
设定的参数决定了输出的特征图的大小：h_out = (h_in - kernel_size + 2*padding)/strides + 1

激活函数

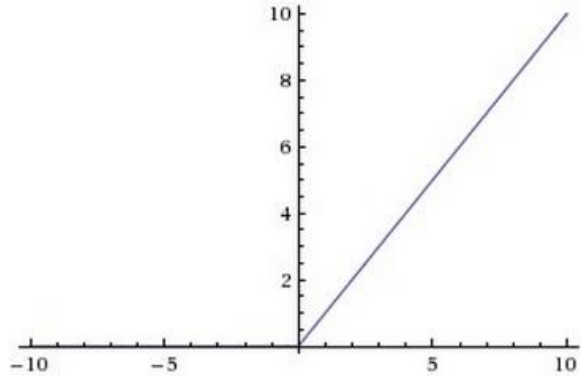
```
sigmoid = nn.Sigmoid()
tanh = nn.Tanh()
relu = nn.ReLU()
```



$$Sigmoid(x) = \frac{1}{1 + e^{-x}}$$



$$Tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

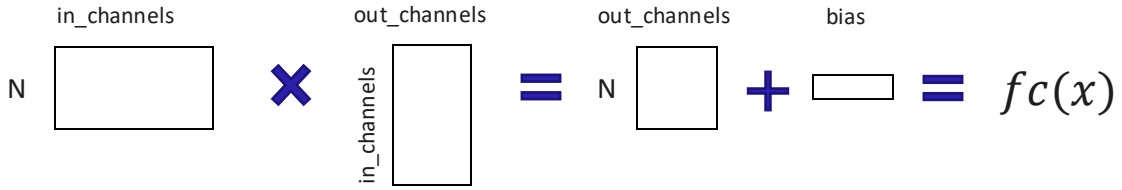


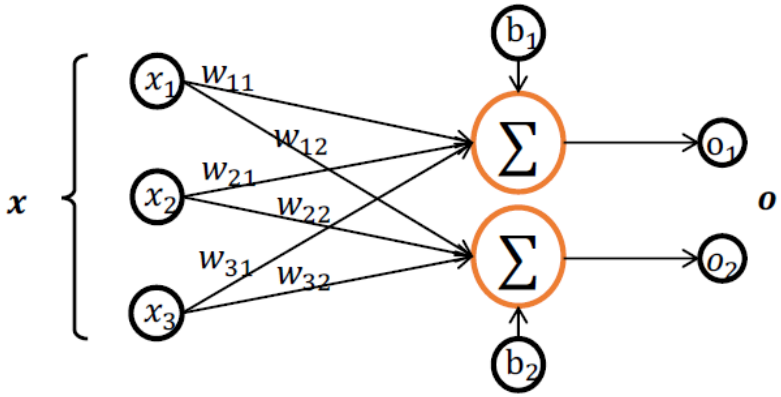
$$ReLU(x) = \max(0, x)$$

在卷积层之后引入激活函数进行非线性变换，可以提高模型的拟合能力。

全连接（Dense）

fc1 = nn.Dense(in_channels, out_channels, weight_init='normal', bias_init='zeros', has_bias=True, activation=None)





y = fc1(x)：

- 输入：Tensor shape(batch_size, in_channels).
- 输出：Tensor shape(batch_size, out_channels).

接受一个2维的张量作为输入，返回一个2维的张量作为输出，2维即batch_size x channels

网络定义

①

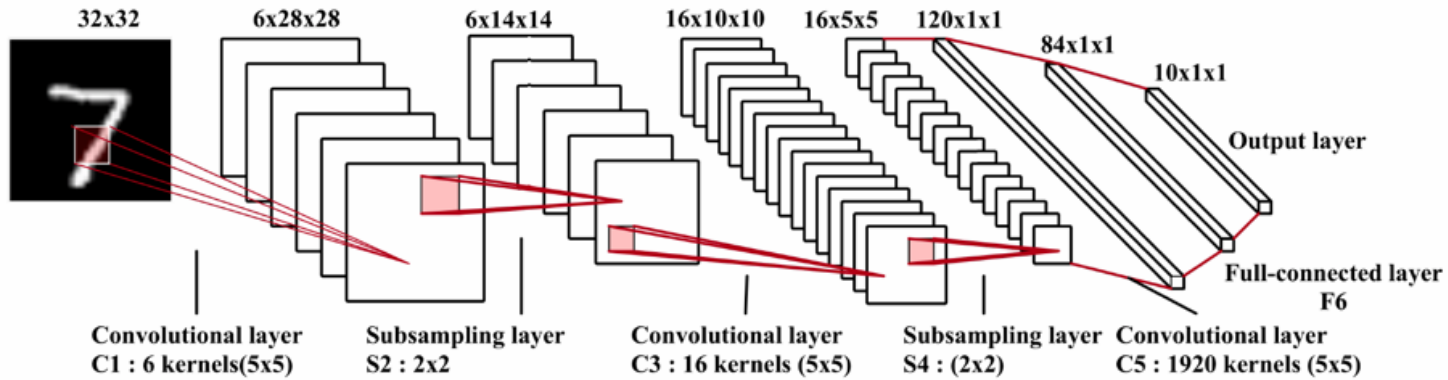
②

```
class LeNet5(nn.Cell):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, stride=1, pad_mode='valid')
        self.conv2 = nn.Conv2d(6, 16, 5, stride=1, pad_mode='valid')
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Dense(400, 120)
        self.fc2 = nn.Dense(120, 84)
        self.fc3 = nn.Dense(84, 10)
```

③

```
    def construct(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)

        return x
```

① 网络定义必须继承基类nn.Cell；
② __init__()中的语句由Python解析执行；
③ construct()中的语句由MindSpore接管，有语法限制；



(a) LeNet-5 network

损失函数（Loss）

loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')

$$Z_i = W \cdot X_i + b$$

$$P_i = \frac{e^{Z_i}}{\sum_j e^{Z_i}}$$

$$l(Z_i, Y_i) = -\log\left(P_{iY_i}\right)$$

对于每个样本N_i，X_i是3D Tensor（CHW），Z_i是3D Tensor，Y_i是真实类别（10类中的一个），P_i是3D Tensor（含10个元素，分别表示属于相应的概率，值域为[0, 1]），$l(Z_i, Y_i)$是标量。

损失函数用于描述模型预测值与真实值的误差，用于指导模型的更新方向。MindSpore支持的损失函数有：L1Loss、MSELoss、SoftmaxCrossEntropyWithLogits、CosineEmbeddingLoss等。

参数解释：

- is_grad，是否仅计算梯度，默认为True；
- sparse，默认为False，为True时对Label数据做one_hot处理；
- reduction，支持mean、sum。

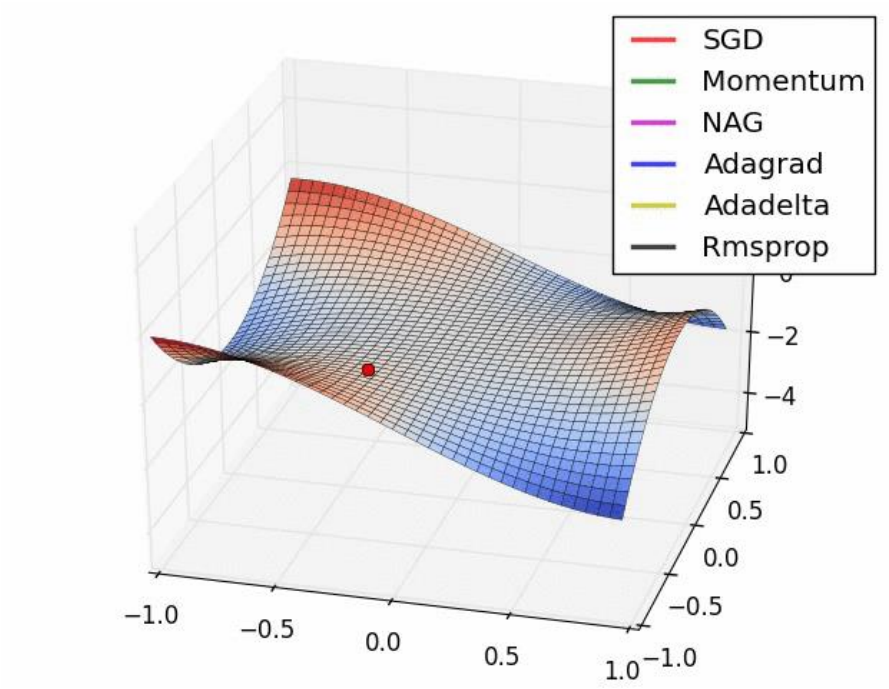
优化器（Optimizer）

```
opt = nn.Momentum(net.trainable_params(), lr, momentum)
```

- 有了优化目标（Loss）之后，我们还需要定义优化的策略，即优化器
- MindSpore支持的优化器有：SGD、Momentum、RMSProp、Adam、AdamWeightDecay、LARS、FTRL、ProximalAdagrad等。

SGD with Momentum:

$$v_{t+1} \leftarrow \rho v_t + \nabla_\theta \mathcal{L}(\theta)$$
$$\theta_j \leftarrow \theta_j - \epsilon v_{t+1}$$

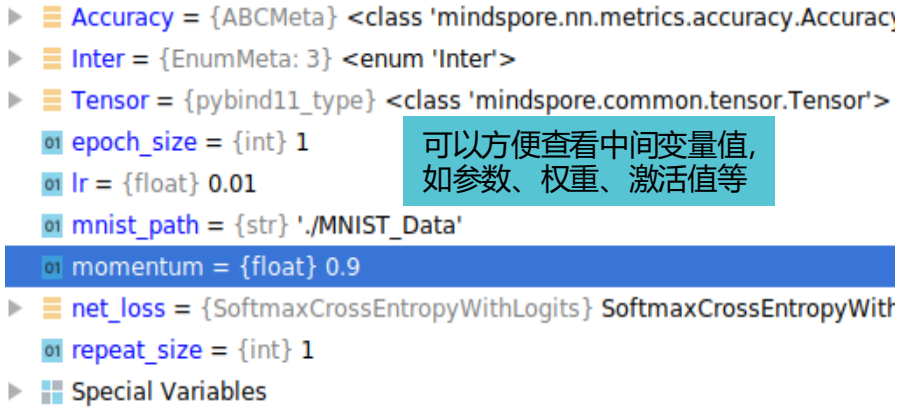


代码调试

MindSpore支持Graph和PyNative 2种运行模式，**PYNATIVE模式易调试，支持Python原生调试方式。**

```
net_loss = SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='
repeat_size = epoch_size  repeat_size: 1
# create the network
network = LeNet5()
# define the optimizer
net_opt = nn.Momentum(network.trainable_params(), lr, momentum)
config_ck = CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)
# save the network model and parameters for subsequence fine-tuning
ckpoint_cb = ModelCheckpoint(prefix="checkpoint_lenet", config=config_ck)
# group layers into an object with training and evaluation features
model = Model(network, net_loss, net_opt, metrics={"Accuracy": Accuracy()})
```

支持Python原生的调试方式，如设置断点



可以方便查看中间变量值，如参数、权重、激活值等

```
def construct(self, x):
    x = self.relu(self.conv1(x))
    x = self.pool(x)
    print(x.shape)
    x = self.relu(self.conv2(x))
    x = self.pool(x)
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.fc2(x)
    x = self.fc3(x)

    return x
```

打印某层算子输出的shape，方便设置下一层算子的参数

GRAPH模式性能高，可调用Print算子打印Tensor或字符串。

```
self.print = P.Print()
self.print(x)
Tensor shape:[[const vector][2, 1]]Int32
val:[[1] [1]]
```

模型调优

- mindspore.train.callback可用于训练/验证过程中执行特定的任务。常用的Callback如下：
  - LossMonitor：监控loss值，当loss值为Nan或Inf时停止训练；
  - SummaryStep：把训练过程中的信息存储到文件中，用于后续查看或可视化展示。
  - ModelCheckpoint：保存模型文件和参数，用于再训练或推理；

```
loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())
# 打印
epoch: 2 step 1875, loss is 0.4737345278263092
```

- mindspore.nn提供了多种Metric评估指标，如accuracy、loss、precision、recall、F1。
  > 定义一个metrics字典/元组，里面包含多种指标，传递给Model；
  > 然后调用model.eval接口来计算这些指标。
  > model.eval会返回一个字典，包含各个指标及其对应的值。

```
metrics={'acc', 'loss'}
model = Model(net, loss, opt, metrics={'acc', 'loss'})
# 输出
{'loss': 0.10531254443608654, 'acc': 0.9701522435897436}
```

模型训练和验证

- mindspore.train.Model提供了高阶模型训练和验证接口。
  - 传入net、loss和opt，并完成Model的初始化；
  - 然后调用train接口，指定迭代次数（num_epochs）、数据集（dataset）、回调（callback）；
- 训练包含两层循环：外层为数据集的多代（epoch）循环，内层为epoch内多步（batch/step）的迭代；

```
def train(data_dir, lr=0.01, momentum=0.9, num_epochs=3):
    ds_train = create_dataset(data_dir)
    ds_eval = create_dataset(data_dir, training=False)

    net = LeNet5()
    loss = nn.loss.SoftmaxCrossEntropyWithLogits(is_grad=False, sparse=True, reduction='mean')
    opt = nn.Momentum(net.trainable_params(), lr, momentum)
    loss_cb = LossMonitor(per_print_times=ds_train.get_dataset_size())

    model = Model(net, loss, opt, metrics={'acc', 'loss'})
    # dataset_sink_mode can be True when using Ascend
    model.train(num_epochs, ds_train, callbacks=[loss_cb], dataset_sink_mode=False)
    metrics = model.eval(ds_eval, dataset_sink_mode=False)
    print('Metrics:', metrics)
```

LeNet5完整代码示例：https://gitee.com/mindspore/course/tree/master/checkpoint

模型保存和加载

- ModelCheckpoint会生成模型（.meta）和Chekpoint（.ckpt）文件，如每个epoch结束时，都保存一次checkpoint。

```
ckpt_cfg = CheckpointConfig(save_checkpoint_steps=steps_per_epoch, keep_checkpoint_max=5)
ckpt_cb = ModelCheckpoint(prefix=ckpt_name, directory='ckpt', config=ckpt_cfg)
model.train(num_epochs, ds_train, callbacks=[ckpt_cb, loss_cb], dataset_sink_mode=dataset_sink)
# 输出:
lenet-1_1875.ckpt
lenet-2_1875.ckpt
lenet-graph.meta
```

- 如果训练中断后想继续做训练，或者加载模型做微调（Fine-tuning），先使用mindspore.train.serialization提供的load_checkpoint、load_param_into_net功能，再行训练。

```
CKPT_1 = 'ckpt/lenet-2_1875.ckpt '
# 加载模型，返回参数字典
param_dict = load_checkpoint(CKPT_1)
# 分别将参数加载到网络和优化器上
load_param_into_net(net, param_dict)
load_param_into_net(opt, param_dict)
```

Checkpoint完整代码示例：https://gitee.com/mindspore/course/tree/master/checkpoint

模型推理

- 使用MindSpore做推理
  - 加载Checkpoint到网络中；
  - 调用model.predict()接口进行推理。

```
CKPT_2 = 'ckpt/lenet_1-2_1875.ckpt'
def infer(data_dir):
    ds = create_dataset(data_dir, training=False).create_dict_iterator()
    data = ds.get_next()
    images = data['image']
    labels = data['label']
    net = LeNet5()
    load_checkpoint(CKPT_2, net=net)
    model = Model(net)
    output = model.predict(Tensor(data['image']))
    preds = np.argmax(output.asnumpy(), axis=1)
```

- 使用其他平台做推理
  - > 加载Checkpoint到网络中；
  - > 调用mindspore.train.serialization提供的export()接口，导出ONNX等格式的模型文件；
  - > 在任何支持ONNX模型文件的平台上进行推理；

```
input = np.random.uniform(0.0, 1.0, size = [1, 3, 224, 224]).astype(np.float32)
export(resnet, Tensor(input), file_name = 'resnet50-2_32.pb', file_format = 'ONNX')
```

部署推理服务

- MindSpore Serving是一个轻量级、高性能的服务模块，旨在帮助MindSpore开发者在生产环境中高效部署在线推理服务。
  - 使用MindSpore完成模型训练；
  - 导出MindSpore模型（BINARY格式）；
  - 使用MindSpore Serving创建推理服务。

```
ms_serving [--help] [--model_path <MODEL_PATH>] [--model_name <MODEL_NAME>]
        [--port <PORT>] [--device_id <DEVICE_ID>]
```

参数名	属性	功能描述	参数类型	默认值	取值范围
--help	可选	显示启动命令的帮助信息。	-	-	-
--model_path <MODEL_PATH>	必选	指定待加载模型的存放路径。	str	空	-
--model_name <MODEL_NAME>	必选	指定待加载模型的文件名。	str	空	-
--port <PORT>	可选	指定Serving对外的端口号。	int	5500	1~65535
--device_id <DEVICE_ID>	可选	指定使用的设备号	int	0	0~7

r0.6及以上版本：https://www.mindspore.cn/tutorial/zh-CN/r0.6/advanced_use/serving.html